

Generazione di Codice

Slides based on material
by Ras Bodik available at
<http://inst.eecs.berkeley.edu/~cs164/fa04>

Status

- We have covered the front-end phases

- Lexical analysis
- Parsing
- Semantic analysis

Il Front-End si concentra sull'analisi del codice sorgente. Generalmente produce una Rappresentazione Intermedia (IR), che è indipendente dalla macchina target.

- Next are the back-end phases

- Optimization
- Code generation

Il Back-End si concentra sulla sintesi del codice target, utilizzando l'IR prodotta dal front-end. È la parte dipendente dall'architettura della CPU (ad esempio, x86, ARM):

- Ottimizzazione (Optimization): Migliora l'IR, rendendola più efficiente in termini di velocità e/o spazio, pur mantenendo la stessa semantica.
- Generazione di Codice (Code Generation): Trasforma l'IR ottimizzata nel codice macchina o assembly specifico che la CPU può eseguire.

- We'll discuss only code generation

Run-time environments

L'Ambiente di Esecuzione è l'insieme delle strutture, delle convenzioni e dei servizi forniti dal sistema operativo, dall'hardware e dalle librerie di supporto del linguaggio che governano l'esecuzione del programma. Il compilatore deve generare codice che sia in grado di interagire correttamente con questo ambiente.

- Before discussing code generation, we need to understand what we are trying to generate
- There are a number of standard techniques for structuring executable code that are widely used

La Generazione di Codice si basa interamente sulle strutture dell'ambiente di esecuzione per sapere come accedere alle variabili (locali, globali, passate per parametro) e come gestire il flusso di controllo delle chiamate/ritorni.

Outline

1 • Management of run-time resources (in particular, memory management)

Questo punto si concentra su come il compilatore gestisce le risorse hardware e software necessarie durante l'esecuzione del programma.

2 • Correspondence between static (compile-time) and dynamic (run-time) structures

Il generatore di codice deve stabilire una corrispondenza tra un simbolo (variabile, funzione) noto staticamente nella Tabella dei Simboli, e la sua posizione dinamica in memoria durante l'esecuzione. Questa corrispondenza è ciò che permette al compilatore di generare le istruzioni di accesso alla memoria corrette.

3 • Storage organization

Questo punto approfondisce la divisione logica e fisica dello spazio di indirizzamento di un processo in esecuzione (il layout della memoria).

Le quattro aree principali:

- Area Codice (Text): Per le istruzioni.
- Area Dati Statici/Globali: Per le variabili globali.
- Stack: Per le chiamate a funzione e le variabili locali automatiche.
- Heap: Per l'allocazione dinamica della memoria.

La conoscenza di come queste aree sono disposte e gestite è la base per generare istruzioni di accesso efficienti.

1. Gestione delle Risorse di Esecuzione (Run-time Resources)

Questo punto si concentra su come il compilatore gestisce le risorse hardware e software necessarie durante l'esecuzione del programma.

- Risorse Hardware: Principalmente i Registri della CPU e la Memoria Centrale. La gestione di queste risorse è fondamentale per l'efficienza. La Generazione di Codice è responsabile di decidere quali variabili allocare nei registri (più veloci) e quando fare spill (salvare) i registri nella memoria.

- Risorse Software: Include l'interazione con il Sistema Operativo per l'I/O, la gestione dei processi, e, in particolare, la Gestione della Memoria (Memory Management).

Il Memory Management qui non si riferisce solo all'uso di malloc o new (che coinvolge l'Heap), ma all'organizzazione generale di tutte le sezioni di memoria di un processo (Code, Static, Stack, Heap).

Memory Management

Run-time Resources

- Execution of a program is initially under the control of the operating system

Quando si lancia un programma, il processo di esecuzione inizia sotto il controllo dell'OS. Il Sistema Operativo deve preparare l'ambiente affinché il codice possa essere eseguito in modo sicuro e isolato dagli altri processi.

- When a program is invoked:

Il Sistema Operativo alloca uno spazio di indirizzamento virtuale per il nuovo processo. Questo spazio è l'illusione di avere una memoria dedicata, anche se fisicamente è condivisa. L'allocazione non è solo per il codice, ma per l'intera organizzazione della memoria (Code, Static, Stack, Heap). Questo spazio allocato costituisce la base dell'Ambiente di Esecuzione (Run-time Environment) su cui si basa la Generazione di Codice.

- The OS allocates space for the program
- The code is loaded into part of the space
- The OS jumps to the entry point (i.e., "main")

Non è detto che la memoria allocata per il programma sia contigua

Una volta che l'ambiente è pronto e il codice è in memoria, l'OS cede il controllo saltando all'indirizzo di memoria iniziale predefinito, noto come Entry Point. Nei programmi in linguaggi di alto livello come C o C++, questo punto è quasi sempre la funzione main(). Tuttavia, il compilatore spesso genera un piccolo pezzo di codice (chiamato codice di startup o runtime library) che viene eseguito prima di main(). Questo codice si occupa di inizializzare l'ambiente di runtime del linguaggio. È dopo l'esecuzione di questo codice di startup che il controllo passa effettivamente alla funzione main().

Il Sistema Operativo legge il file eseguibile dal disco e carica le istruzioni binarie (il codice macchina) nella sezione Code dello spazio di memoria allocato. Inoltre, vengono caricate le variabili globali e statiche nelle loro rispettive sezioni, spesso inizializzandole con i valori predefiniti specificati nel codice sorgente.

Memory Layout

Memory

del processo del programma

Code

Other Space

Questa è la sezione in cui risiedono le istruzioni macchina generate dal compilatore:

- Contenuto: Le istruzioni del programma compilato (il codice eseguibile).
- Caratteristiche: Questa sezione è tipicamente sola lettura (read-only) e statica, il che significa che la sua dimensione è fissata durante la compilazione o il linking (collegamento) e non cambia durante l'esecuzione.
- Importanza per il Compilatore: Il generatore di codice popola questa sezione. Poiché il codice è statico, l'indirizzo di qualsiasi funzione o blocco di codice è noto e fisso, permettendo al compilatore di generare istruzioni di salto (jump) e chiamata (call) con indirizzi precisi.

Notes

Nei diagrammi, vediamo spesso la memoria divisa da linee nette in sezioni fisse (Code, Static, Stack, Heap). Queste linee indicano la separazione logica e le diverse politiche di gestione per i dati (ad esempio, lo Stack è LIFO, l'Heap è dinamico e libero). Il compilatore deve rispettare queste aree logiche per generare gli indirizzi di accesso corretti. Ad esempio, una variabile locale deve essere indirizzata rispetto al Puntatore allo Stack (SP) o al Puntatore Base (BP), non rispetto all'indirizzo globale.

- By tradition, pictures of machine organization have:

- Lines delimiting areas for different kinds of data

- These pictures are simplifications

- E.g., not all memory needs be contiguous

Questi diagrammi sono semplificazioni:

- Memoria Virtuale: I sistemi operativi moderni utilizzano la Memoria Virtuale. Lo spazio di indirizzamento di un processo (quello che il compilatore vede) è un indirizzamento logico che va da 0 a N.
- Paging: Il sistema operativo mappa questo spazio virtuale in pagine fisiche che, nel mondo reale, non devono essere contigue. Possono essere sparse in diverse aree della RAM o addirittura scambiate su disco (in swap).
- L'Indipendenza del Compilatore: Il bello di questa astrazione è che il compilatore non deve preoccuparsi della non contiguità fisica. Il suo unico compito è generare gli indirizzi virtuali corretti. L'OS e l'Hardware (tramite la MMU - Memory Management Unit) si occupano della traduzione degli indirizzi virtuali in fisici.

What is Other Space?

- Holds all data for the program
- Other Space = Data Space
- Compiler is responsible for:
 - Generating code
 - Organizing use of the data area

Organizzazione dell'uso dell'Area Dati: Decidere dove posizionare ogni pezzo di dato (globale? sullo Stack? sull'Heap?) e generare il codice che sa come accedere a tale posizione.

Code Generation Goals

- **Two goals:**

- **Correctness**

Il codice macchina o assembly generato deve essere semanticamente equivalente al programma sorgente originale. Deve produrre lo stesso output e avere lo stesso comportamento logico del codice scritto dal programmatore. Se un compilatore genera codice veloce ma sbagliato, è inutile. La correttezza è la prima preoccupazione e solitamente la parte più facile da assicurare se si segue una traduzione diretta (anche se inefficiente) dalla Rappresentazione Intermedia (IR).

- **Speed**

Il codice generato deve essere efficiente in termini di tempo di esecuzione e di utilizzo delle risorse (memoria, registri). Un programma generato deve eseguire il minor numero possibile di cicli di clock e accedere alla memoria il meno possibile. Il compilatore deve sfruttare le caratteristiche specifiche della CPU target, come:

- Allocations dei Registri: Mantenere i valori più usati nei registri della CPU (che sono velocissimi) invece che nella memoria (che è lenta).
- Selezione delle Istruzioni: Scegliere l'istruzione assembly più veloce per una data operazione.
- Ordinamento delle Istruzioni: Riordinare le istruzioni per massimizzare l'uso delle pipeline della CPU.

- **Most complications in code generation come from trying to be fast as well as correct**

Assumptions about Execution

1. Execution is sequential; control moves from one point in a program to another in a well-defined order (e.g. a single thread)

Questa assunzione ignora la complessità del multithreading o dell'esecuzione parallela, che richiederebbe meccanismi di sincronizzazione molto più complessi. Assumere un singolo thread semplifica enormemente la gestione dello stato e delle risorse.

→ Quando una procedura (funzione) viene chiamata, il controllo torna al punto immediatamente successivo alla chiamata quando la procedura termina.

2. When a procedure is called, control eventually returns to the point immediately after the call

Activations

funzione/metodo/procedura

An invocation of procedure *P* is an activation of *P*

Un'attivazione è semplicemente una singola esecuzione (o invocazione) di una specifica procedura (funzione o metodo) in un dato momento.

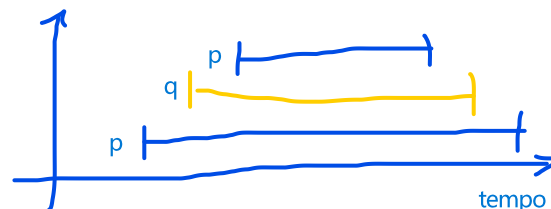
Se una funzione ricorsiva *P* chiama se stessa, si avranno attivazioni multiple e distinte di *P* in esecuzione contemporaneamente. Il compilatore deve garantire che ogni attivazione abbia il proprio spazio dati isolato (il Record di Attivazione).

The *lifetime* of an activation of *P* is

- All the steps to execute *P*
- Including all the steps in procedures *P* calls

La lifetime (durata) di un'attivazione *P* è il tempo che intercorre dall'inizio della sua esecuzione fino al momento in cui restituisce il controllo al chiamante. La durata include:

- Esecuzione del Corpo di *P*: Tutte le istruzioni eseguite direttamente all'interno della procedura *P*.
- Tutte le Procedure Chiamate da *P*: Se *P* chiama la procedura *Q*, l'esecuzione di *Q* (e di tutte le procedure che *Q* chiama a sua volta) è inclusa nella lifetime dell'attivazione di *P*.



Le vite di attivazioni di procedure possono essere innestate

Lifetimes of Variables

- The *lifetime* of a variable x is the portion of execution in which x is defined

La Lifetime di una variabile x è il periodo di tempo (o la porzione di esecuzione) in cui x è definita e ha una cella di memoria assegnata per conservare il suo valore (la sua vita dura dal momento della dichiarazione fino all'uscita dallo scope della dichiarazione).

- Note that
 - Lifetime is a dynamic (run-time) concept
 - Scope is a static concept

Activation Trees

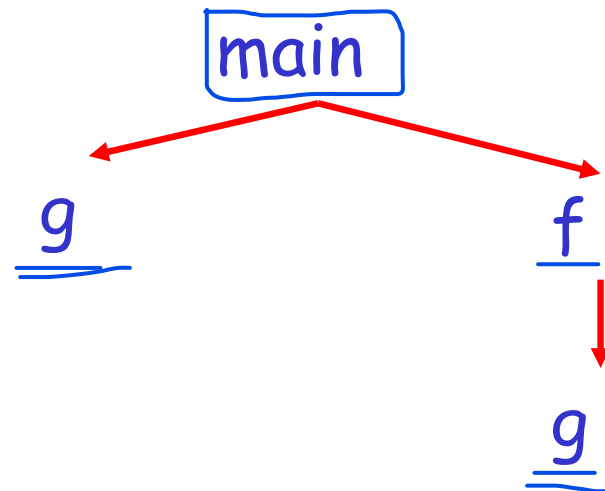
Quando una procedura (funzione) viene chiamata, il controllo torna al punto immediatamente successivo alla chiamata quando la procedura termina.

- Assumption (2) requires that when P calls Q , then Q returns before P does
- Lifetimes of procedure activations are properly nested
- Activation lifetimes can be depicted as a tree

Example

Rappresentare solo le attivazioni di funzioni

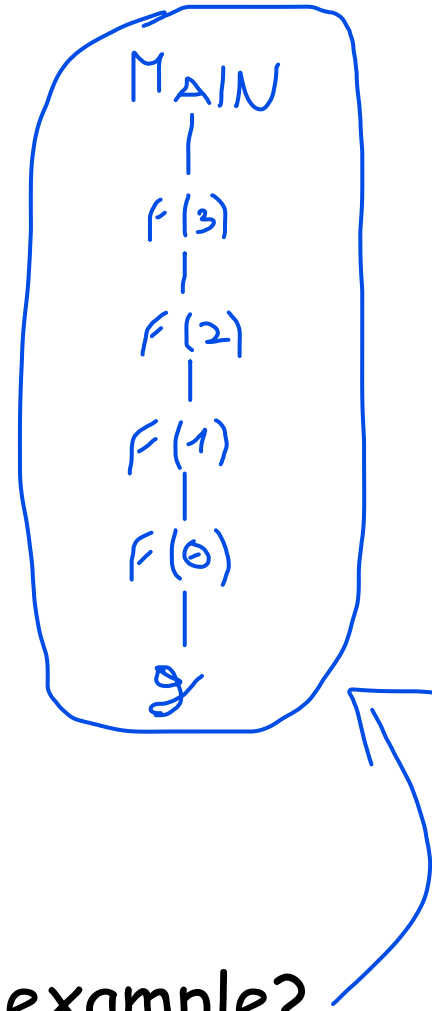
```
class Main {  
    int g() { return 1; }  
    int f() { return g(); }  
    void main() { g(); f(); }  
}
```



Example 2

```
class Main {  
    int g() { return 1; }  
    int f(int x) {  
        if (x == 0) { return g(); }  
        else { return f(x - 1); }  
    }  
    void main() { f(3); }  
}
```

start →



What is the activation tree for this example?

Notes

- The activation tree depends on run-time behavior

L'Albero delle Attivazioni è un concetto dinamico. Non può essere costruito o determinato completamente a compile-time. La sua struttura (quali funzioni vengono chiamate, e quante volte) dipende interamente dalle decisioni prese durante l'esecuzione del programma.

- The activation tree may be different for every program input

Poiché il comportamento run-time dipende dall'input, anche l'Albero delle Attivazioni può cambiare.

- Since activations are properly nested, a stack can track currently active procedures

La proprietà cruciale delle attivazioni procedurali è che sono annidate (properly nested). Questo è il concetto di Last-In, First-Out (LIFO). Una funzione chiamata deve sempre terminare prima che possa riprendere l'esecuzione la funzione che l'ha chiamata. Questa proprietà LIFO rende lo Stack la struttura dati perfetta per tracciare le procedure attualmente attive. Quando una funzione è attivata (chiamata), il suo Record di Attivazione viene messo in cima allo Stack (PUSH). Quando una funzione termina (ritorna), il suo Record di Attivazione viene tolto dalla cima dello Stack (POP).

Example

```
class Main {  
    int g() { return 1; }  
    int f() { return g(); }  
    void main() { g(); f(); }  
}
```

main

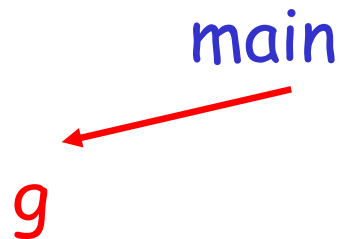
Lo stack cresce verso il basso (indirizzi alti nella parte alta dello stack e indirizzi più bassi sulla cima dello stack)

Stack

main

Example

```
class Main {  
    int g() { return 1; }  
    int f() { return g(); }  
    void main() { g(); f(); }  
}
```



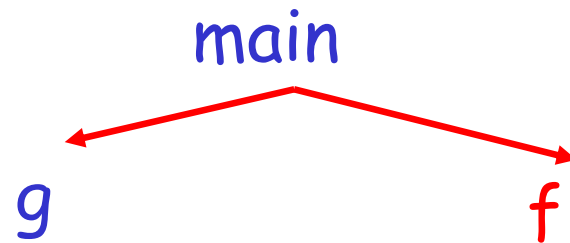
Stack

main

g

Example

```
class Main {  
    int g() { return 1; }  
    int f() { return g(); }  
    void main() { g(); f(); }  
}
```



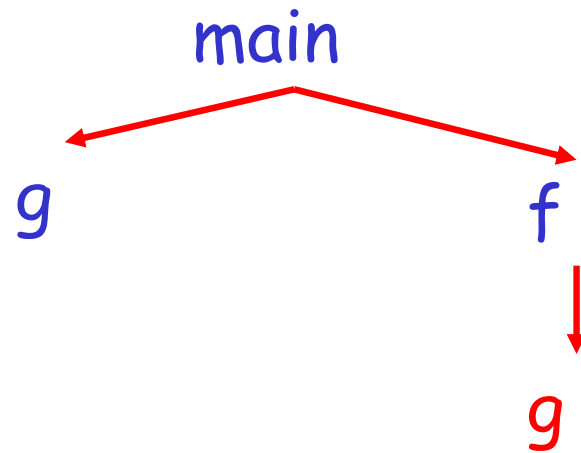
Stack

main

f

Example

```
class Main {  
    int g() { return 1; }  
    int f() { return g(); }  
    void main() { g(); f(); }  
}
```



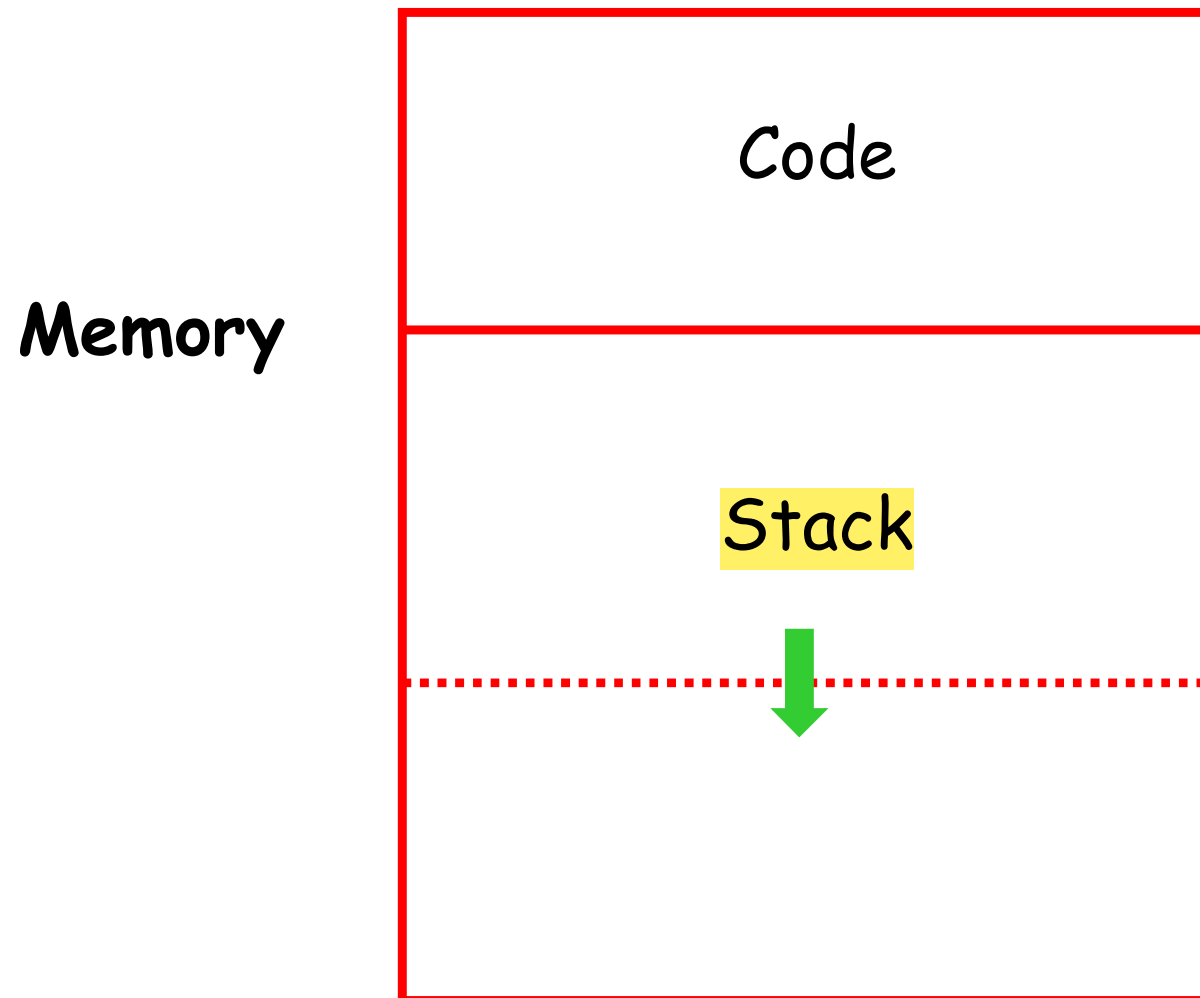
Stack

main

f

g

Revised Memory Layout



Activation Records

Il Record di Attivazione è una struttura dati logica (e fisica) che viene allocata sullo Stack all'inizio dell'esecuzione di una funzione e deallocata alla sua fine. Il suo scopo è contenere tutti i dati e i metadati necessari per gestire quella specifica attivazione della funzione.

- The information needed to manage one procedure activation is called an *activation record (AR)* or *frame*
- If procedure f calls g , then g 's activation record contains a mix of info about f and g .

What is in g 's AR when f calls g ?

- f is “suspended” until g completes, at which point f resumes. g 's AR contains information needed to resume execution of f .



- Indirizzo di Ritorno: L'indirizzo di memoria esatto all'interno del codice di f dove l'esecuzione deve riprendere subito dopo l'istruzione di CALL a g . Questo è il dato fondamentale per la ripresa di f .
- Stato della Macchina Salvato: Il contenuto dei registri della CPU che f sta usando e che g potrebbe sovrascrivere. Le convenzioni di chiamata (calling conventions) stabiliscono quali registri devono essere salvati da f (caller-saved) e quali da g (callee-saved). Questo garantisce che quando f riprende, il suo contesto di calcolo (i suoi dati temporanei nei registri) sia intatto.

- g 's AR may also contain:
 - g 's return value (needed by f)
 - Actual parameters to g (supplied by f)
 - Space for g 's local variables

— Valori Temporanei: Spazio per i risultati intermedi di calcoli complessi all'interno di g .

The Contents of a Typical AR for g

- Pointer to the previous activation record
 - The *control link*; points to AR of caller of g
- Machine status prior to calling g
 - Contents of registers & program counter

Salva il contesto di esecuzione di f . Contiene i valori dei registri della CPU (e l'indirizzo di Ritorno) di f prima che g inizi la sua esecuzione.

- Space for g 's return value
- Actual parameters
- Local variables
- Other temporary values

Spazio per i risultati intermedi di espressioni complesse che non possono essere tenuti nei registri per mancanza di spazio o per la necessità di salvarli attraverso una chiamata di funzione.

Example 2, Revisited

```
class Main {  
    int g() { return 1; }  
    int f(int x) {  
        if (x == 0) { return g(); }  
        else { return f(x - 1); (**) }  
    }  
    void main() { f(3); (*) }  
}
```

AR for f:

result	Valore di ritorno
argument	Parametro Attuale x
control link	Puntatore ad AR del chiamante
return address	Indirizzo di Ritorno

Lo spazio riservato per memorizzare il valore restituito da questa specifica attivazione di f prima che termini. Quando un'attivazione di f termina, scrive il risultato (ottenuto dalla chiamata a g() o da un'altra chiamata ricorsiva a f) in questo slot o in un registro predefinito. Il chiamante leggerà questo risultato.

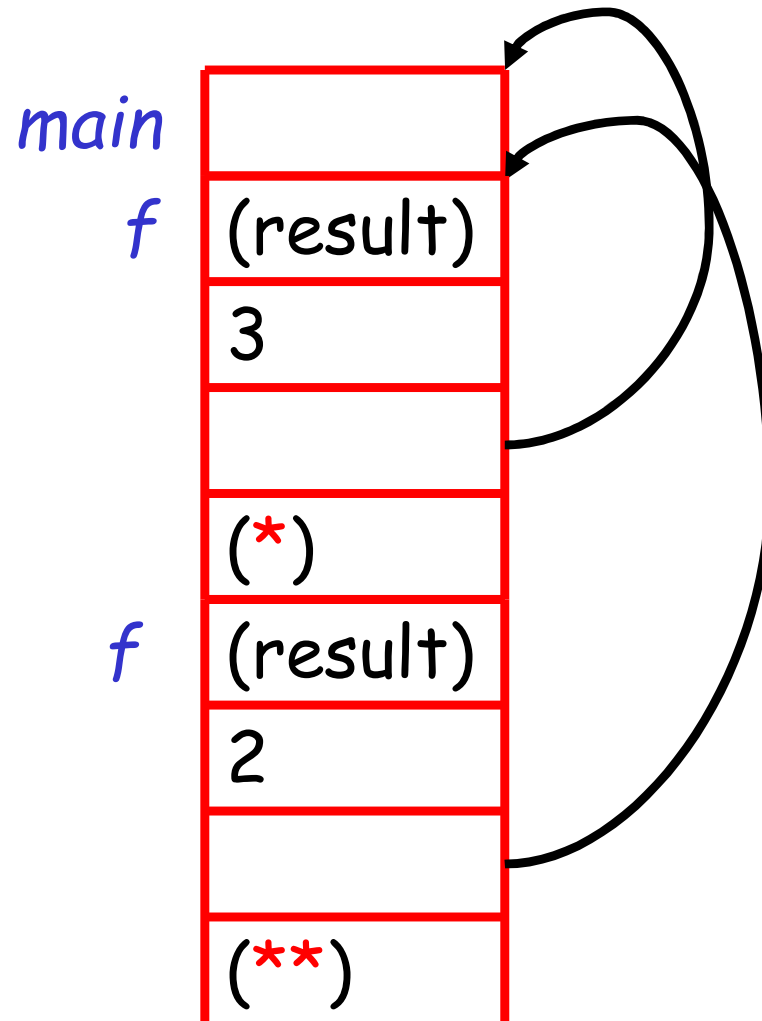
Il valore intero specifico (x) passato a questa particolare attivazione di f. Il generatore di codice genera istruzioni per accedere a questo campo (tipicamente tramite un offset fisso dal Base Pointer) ogni volta che la variabile x viene letta all'interno di f.

L'indirizzo di memoria dell'istruzione immediatamente successiva alla chiamata che ha attivato l'AR corrente.

- Se main chiama f(3) (*): L'indirizzo salvato qui è l'istruzione dopo la chiamata f(3); in main.
- Se f(2) chiama f(1) (**): L'indirizzo salvato qui è l'istruzione dopo la chiamata f(x - 1); all'interno dell'AR di f(2).
- Viene salvato durante l'istruzione di CALL e letto e utilizzato per il salto (JUMP) dall'istruzione di RET (ritorno) al completamento dell'attivazione.

Un puntatore all'inizio del Record di Attivazione del chiamante. Viene salvato nel prologo di f. È fondamentale per ripristinare il Frame Pointer (Base Pointer) all'uscita di f. Assicura che, dopo che un AR è stato tolto, il Base Pointer indichi correttamente l'inizio del Frame del chiamante.

Stack After Two Calls to f



Notes

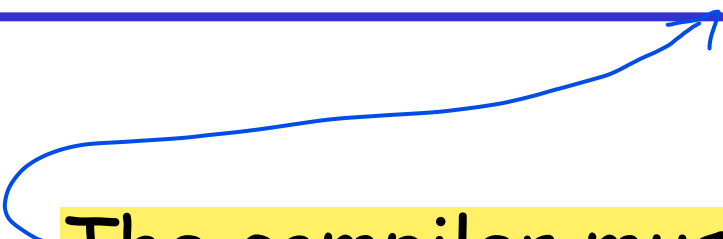
- main has no argument, no return address and its result is never used; its AR is uninteresting
- (*) and (**) are return addresses of the invocations of f
 - The return address is where execution resumes after a procedure call finishes
- This is only one of many possible AR designs
 - Would also work for C, Pascal, FORTRAN (from 90, not 77 that disallows recursion), etc.

1. Determinazione Statica del Layout
Il compilatore, in fase di compile-time (statica), deve:
Calcolare gli Offset: Per ogni variabile locale, parametro, link di controllo e temporaneo di una funzione, il compilatore deve determinare il suo offset (spostamento) fisso rispetto al Base Pointer (BP) o Frame Pointer (FP) del Record di Attivazione (AR).
Determinare la Dimensione: Deve calcolare la dimensione totale fissa che il Record di Attivazione richiederà sullo Stack.
Nonostante l'AR venga allocato dinamicamente a run-time, il suo layout interno (la posizione relativa di tutti i suoi campi) è statico e predeterminato dal compilatore.

2. Generazione del Codice di Accesso Corretto

Una volta noto il layout, il generatore di codice deve produrre istruzioni assembly che accedano a questi campi usando l'offset calcolato. Ad esempio, per accedere alla variabile locale x, il codice generato non sarà un indirizzo assoluto, ma uno relativo rispetto a Base Pointer.

The Main Point



The compiler must determine, at compile-time, the layout of activation records and generate code that correctly accesses locations in the activation record

Quindi

Thus, the AR layout and the code generator must be designed together!

Example

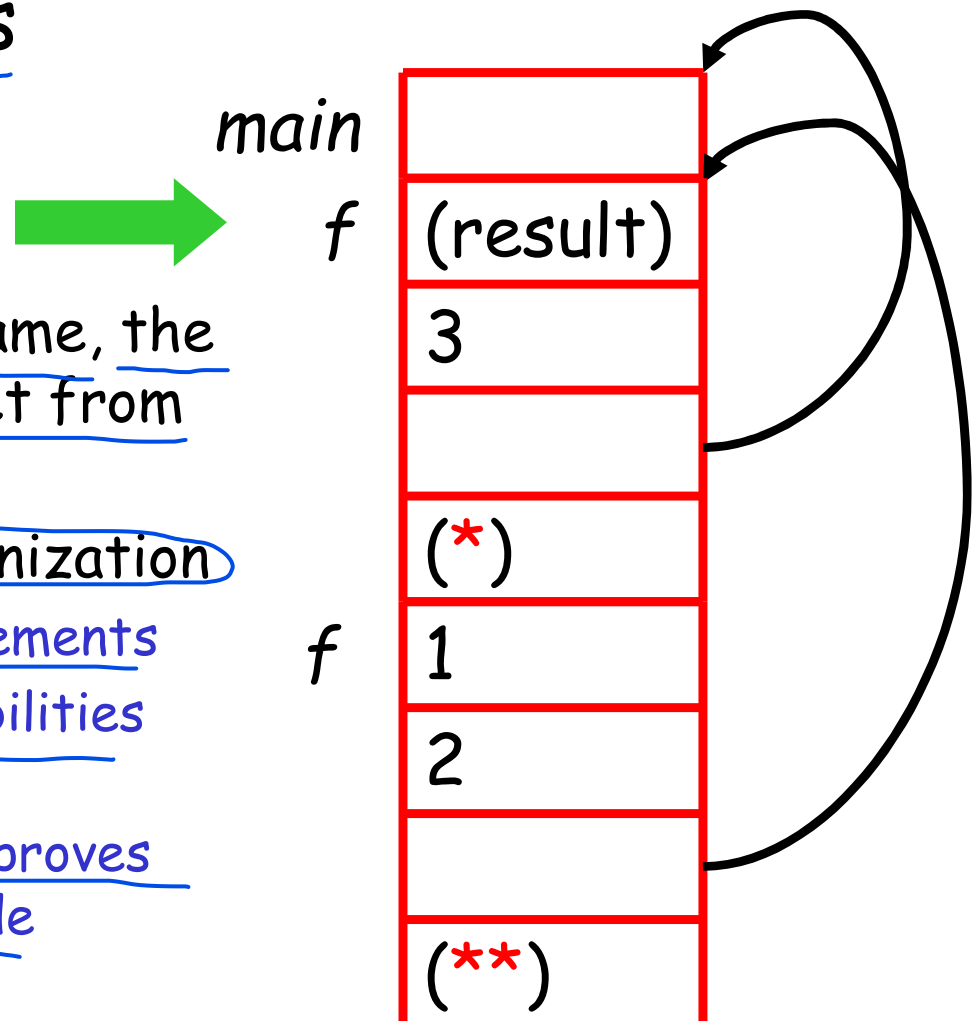
Prima di disallocare la funzione chiamata, il chiamante deve prendere il risultato del chiamato: il risultato é posizionato ad un fixed-offset del frame

The picture shows the state when the call to 2nd invocation of *f* returns

As the return value is 1st in a frame, the caller can find it at a fixed offset from its own frame

This is not the only possible organization

- Can rearrange order of frame elements
- Can divide caller/callee responsibilities differently
- An organization is better if it improves execution speed or simplifies code generation



Ci possono essere delle organizzazioni migliori che migliorano l'efficienza o la semplicità

Global variables

A differenza delle variabili locali, dove ogni attivazione di funzione ha la sua copia nello Stack, esiste una sola copia di ogni variabile globale. Tutte le funzioni, ovunque vengano chiamate, accedono a questo singolo spazio di memoria.

- All references to a global variable point to the same element

- Can't store a global in an activation record

Poiché devono esistere per l'intera durata del programma e non cambiano posizione, le variabili globali vengono assegnate a un indirizzo di memoria fisso (compile-time o link-time).

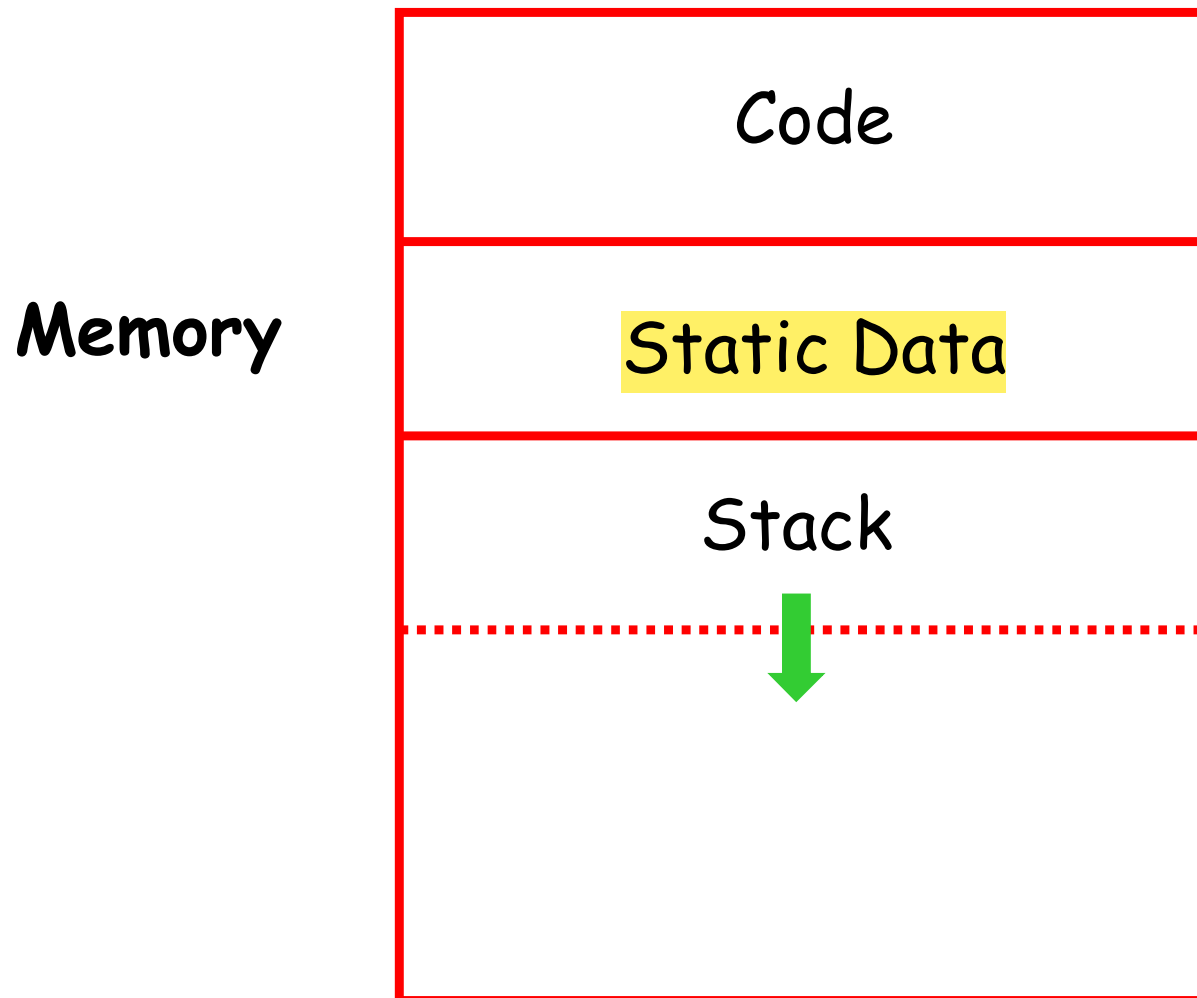
- Globals are assigned a fixed address once

- Variables with fixed address are “statically allocated”

Le variabili staticamente allocate risiedono nella sezione Dati Statici/Globali dello spazio di indirizzamento virtuale del processo.

- Depending on the language, there may be other statically allocated values

Memory Layout with Static Data



Variables declared in outer scopes

- References to a variable declared in an outer scope
 - Should point to a variable stored in another activation record
 - Also the execution of a block {...} has its own activation record
- To which activation record?
 - According to the “most closely nested” rule, an activation record should point to the most recent activation record of its immediately enclosing scope

i Link di Accesso creano una catena (la Catena Statica) che ignora la sequenza dinamica delle chiamate (Control Link) e segue invece la struttura annidata del codice sorgente, garantendo che le variabili vengano risolte in base alla regola "most closely nested".

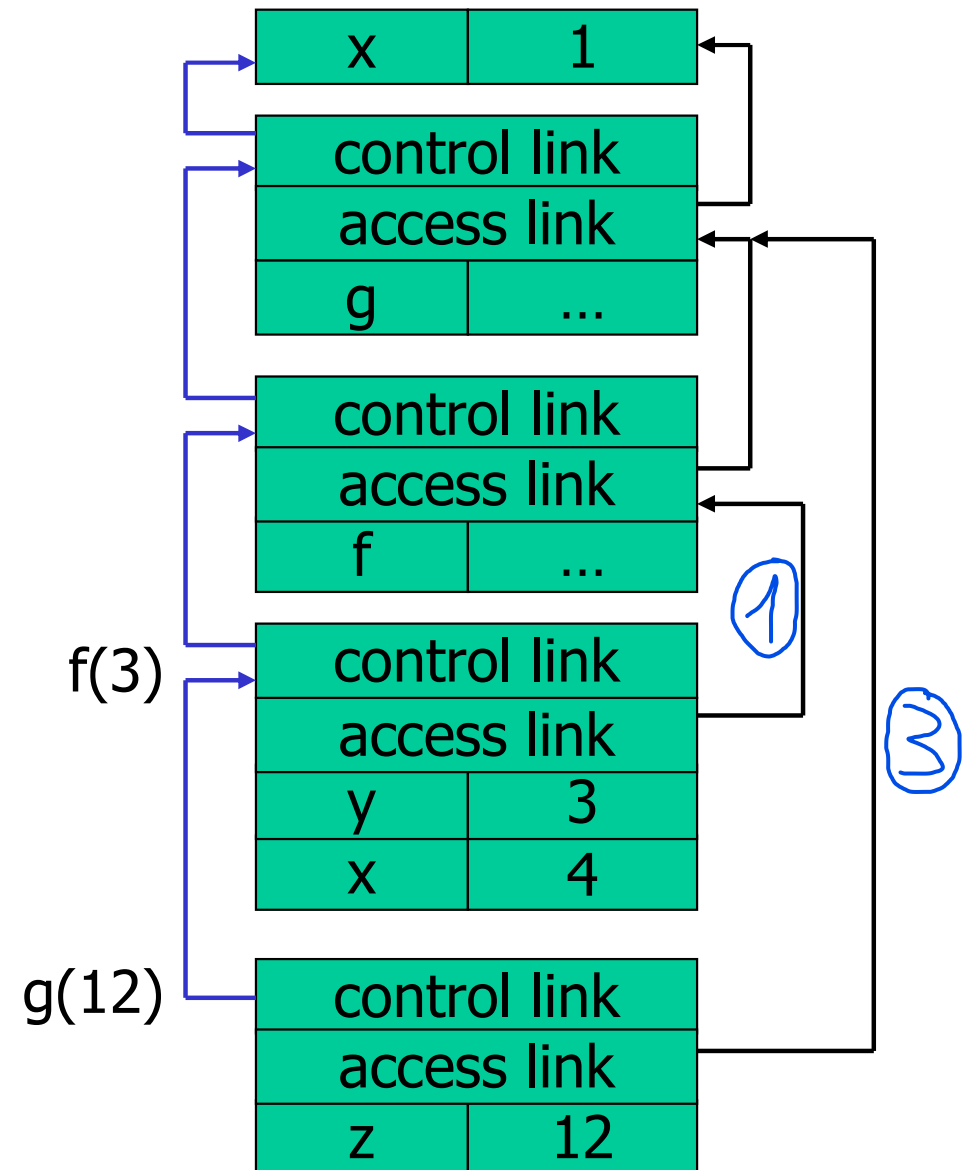
Static scope with access links

Access Link: Collega l'attivazione corrente all'AR dell'attivazione della funzione che la contiene staticamente (lessicalmente). Serve per l'accesso a variabili non locali.

```
int x = 1;  
{ function g(z)={ return x+z };  
  { function f(y)=  
    { int x = y+1;  
      return g(y*x) };  
    f(3);
```

Use access link to find variables defined in outer scopes:

- Access link is always set to frame of enclosing syntactical block
- For function body, this is block that contains function declaration



Setting the access link

La determinazione dell'Access Link si basa sulla relazione statica (lessicale) tra la funzione chiamante e la funzione chiamata, e sul loro livello di annidamento (nesting level). Assumiamo che l'AR corrente sia quello della funzione chiamante (P), e che si stia per creare il nuovo AR per la funzione chiamata (Q).

- **Value of the ACCESS LINK of a new activation record to be created is established as follows:**

① Blocco Interno o Funzione Contenuta (Annidamento Diretto)
An inner block is entered or a function declared in the current scope is called: Il codice sta entrando in un blocco interno o sta chiamando una funzione (Q) che è stata dichiarata direttamente nello scope della funzione chiamante (P).

- **ACCESS LINK = current AR** Access Link di Q = AR Corrente (l'AR di P)

② Chiamata Ricorsiva o Chiamata ad un Compagno di Scope
A function calls itself recursively or calls another function declared in the enclosing syntactical block:

Valore del Access Link dell'AR Corrente (AR di P)
ACCESS LINK = value of ACCESS LINK of the current AR

③ Chiamata a Funzione Esterna a P (Distanza > 1)
In general, call to a function outside the current scope:

Segui la Catena di Access Link per la differenza tra i livelli di annidamento statico.
ACCESS LINK = follow the chain of ACCESS LINKs for the difference between current nesting level and that of function declaration (such a difference is 1 in previous case)

Scenario: Una funzione P chiama un'altra funzione Q, e sia P che Q sono state dichiarate nello stesso blocco sintattico contenitore (lo stesso scope statico A).
Significato: Poiché P e Q sono "sorelle" a livello di dichiarazione, il loro scope esterno lessicale è lo stesso blocco A.

Una funzione P chiama sé stessa (chiamata ricorsiva) oppure Una funzione P chiama un'altra funzione Q che è stata dichiarata nello stesso scope che contiene P.

La funzione chiamata (Q) non è stata dichiarata immediatamente nello scope di P, ma in uno scope più esterno (es. k livelli più esterni, dove $k > 1$).

Se P è al livello N e Q è stata dichiarata al livello N-k, il Link di Accesso di Q deve puntare all'attivazione di livello N-k. Il compilatore genera codice che attraversa la catena di Link di Accesso dell'AR di P un numero di volte pari a $\text{livello}(P) - \text{livello}(Q)$.

Setting the access link

- Value of the ACCESS LINK of a new activation record to be created is established as follows:
 - An inner block is entered or a function declared in the current scope is called:
 - ACCESS LINK = (address of ACCESS LINK in) current AR
 - A function calls itself recursively or calls another function declared in the enclosing syntactical block:
 - ACCESS LINK = value of ACCESS LINK of the current AR
 - In general, call to a function outside the current scope:
 - ACCESS LINK = follow the chain of ACCESS LINKs for the difference between current nesting level and that of function declaration (such a difference is 1 in previous case)

L'Access Link di ogni AR punta all'indirizzo dell'Access Link dell'AR a cui punto, cioè dove proprio si trova in memoria l'Access link dell'AR a cui punto



Non tutto a run-time é Stack.

Heap Storage

L'Heap è la regione di memoria dedicata alla gestione dei dati la cui durata (lifetime) è indipendente dalla sequenza di chiamate a funzione e dai Record di Attivazione (AR) sullo Stack.

Un valore che sopravvive (outlives) alla procedura che lo ha creato non può essere conservato nel Record di Attivazione (AR) di quella procedura.

- A value that outlives the procedure that creates it cannot be kept in the AR

Se la funzione `foo()` alloca un valore `X` nel suo AR (sullo Stack) e poi restituisce un puntatore a `X`, non appena `foo()` termina, il suo AR viene deallocato (tolto dallo Stack). Il puntatore restituito punterebbe ora a una regione di memoria non valida o che potrebbe essere sovrascritta dalla successiva chiamata a funzione.

- `Bar foo() { return new Bar }`
→ The Bar value must survive deallocation of foo's AR

- Languages with dynamically allocated data use a heap to store dynamic data

I linguaggi che permettono l'allocazione dinamica di memoria usano l'Heap per conservare questi dati.

- Heap: È una grande area di memoria che non segue la regola LIFO dello Stack. La sua allocazione e deallocazione sono esplicite (tramite istruzioni nel codice) o gestite automaticamente dal Garbage Collector.
- Puntatori: La funzione `foo()` alloca l'oggetto `Bar` nell'Heap, ma all'interno del suo AR (sullo Stack) memorizza e restituisce solo un puntatore (un indirizzo) a quella posizione nell'Heap.

- Stack: La deallocazione è automatica e deterministica. Quando l'esecuzione esce da un scope (cioè, quando una funzione ritorna), il suo Record di Attivazione (AR) viene tolto dallo Stack e la memoria viene immediatamente liberata.
- Heap: La deallocazione è gestita dal programmatore (es. free in C) o, come descritto qui, da un Garbage Collector.

Garbage Collection

Il problema principale dell'Heap è che, a differenza dello Stack, non ha una regola automatica come LIFO per la deallocazione.

- The ARs in the stack are deallocated when the control exits from the corresponding scope
- What about data in the Heap?
 - They can be removed when they become “Garbage”:
 - At a given point in the execution of a program P , a memory location m is garbage if no continued execution of P from this point can access location m

La Garbage Collection è il processo di recupero automatico della memoria occupata dai dati garbage nell'Heap.

Una locazione di memoria m è considerata garbage se non esiste nessun modo per il programma di accedere a m in qualsiasi esecuzione futura, partendo dal punto corrente. I dati sono accessibili solo se sono referenziati (puntati) da variabili attive nello Stack (ARs attivi) o da altre locazioni non-garbage nell'Heap. Una volta che l'ultimo puntatore ad un oggetto nell'Heap scompare (ad esempio, una variabile locale che conteneva il puntatore viene tolta dallo Stack), quell'oggetto è “dangling” e diventa garbage.

Garbage Collection:

- Detect garbage during program execution
- GC invoked when more memory is needed
- Decision made by run-time system, not program

- Rilevamento: Il GC rileva la garbage durante l'esecuzione del programma.
- Invocazione: Il GC è un servizio del run-time system e viene invocato automaticamente quando è necessario liberare più memoria, tipicamente quando l'Heap si sta riempiendo e il programma richiede una nuova allocazione.
- Decisione di Deallocazione: La decisione di deallocare m è presa dal sistema run-time (l'algoritmo di GC), non dal programma.

1. Rischio di Memory Leak: Se il programmatore dimentica di chiamare free(), la memoria rimane allocata anche se non è più raggiungibile (diventa garbage). Questo porta a Memory Leak, dove il programma consuma gradualmente sempre più memoria, fino al potenziale esaurimento delle risorse.
2. Rischio di Puntatori Pendenti (Dangling Pointers): Se il programmatore chiama free() troppo presto, altri puntatori che ancora si riferiscono a quella locazione di memoria ora liberata diventano puntatori pendenti (dangling pointers). L'uso successivo di questi puntatori può portare a:
- Corruzione dei Dati: Sovrascrivere i dati di un nuovo oggetto che l'allocatore ha riposizionato nella memoria liberata.
 - Vulnerabilità alla Sicurezza: Esporre dati sensibili che si trovavano nella memoria liberata.
 - Crash del Programma: Tentare di leggere o scrivere in memoria non valida.
- Il passaggio chiave è che la GC libera il programmatore dalla responsabilità della deallocazione, ma introduce la complessità di dover interrompere il programma per eseguire l'operazione di pulizia (la "pausa" del GC).

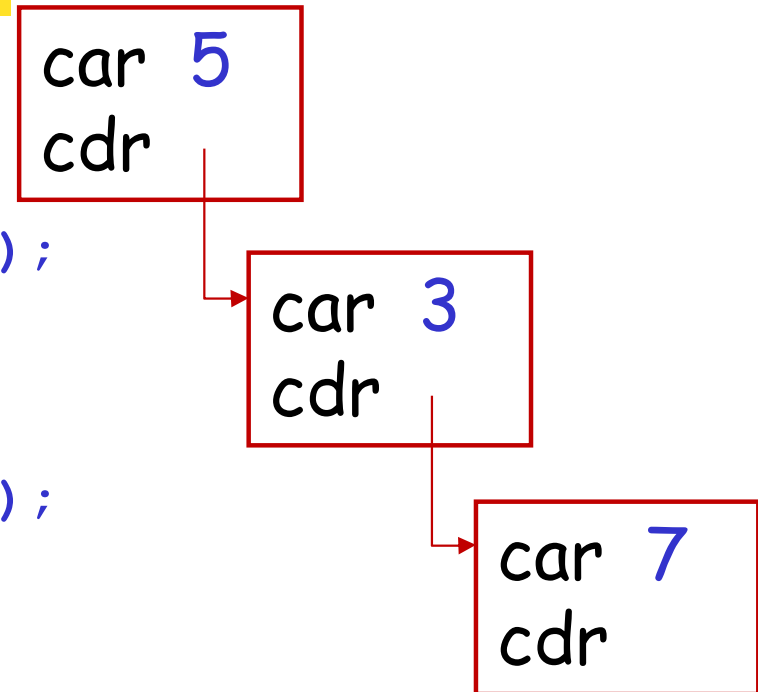
Why Garbage Collection?

In some languages, deallocation is under the responsibility of the programmer

- Example of deallocation in C:

```
for(ptr=lst; ptr!=NULL; ){  
    if (ptr->car == x) return(ptr->cdr);  
    else ptr = ptr->cdr;}  
  
for(ptr=lst; ptr!=NULL; ){  
    if (ptr->car == x) return(ptr->cdr);  
    else{previous = ptr;  
         ptr = ptr->cdr;  
         free(previous);}}
```

Si deve decidere se usare la prima implementazione o la seconda



Problema: in caso di condivisione (più puntatori alla stessa locazione) la cella non può essere effettivamente liberata!!

Problem: in the case of sharing (more pointers to the same location) the cell cannot be actually freed !!

La Garbage Collection risolve il problema della condivisione in modo automatico.

- Il GC non si basa sulla singola istruzione free().
- Il GC attende che tutti i riferimenti a un blocco di memoria scompaiano prima di dichiararlo garbage e recuperarlo. L'approccio tipico è determinare la raggiungibilità (se è raggiungibile dallo Stack, non è garbage).

In sintesi, la Garbage Collection è stata inventata per eliminare gli errori umani derivanti dalla deallocazione manuale, in particolare nel contesto dei dangling pointers e della gestione della memoria condivisa, garantendo la correttezza del programma a costo di un overhead run-time.

Il GC, rispetto al programmatore, ha una visione d'insieme e lo capisce facilmente se una locazione di memoria è garbage, cosa che per il programmatore è più difficile

- Vantaggio Principale: Correttezza. Il Mark-and-Sweep garantisce che la memoria venga liberata solo quando non è più possibile accedervi, risolvendo il problema della condivisione senza rischio di dangling pointers.
- Svantaggio Principale: Frammentazione. Dopo la pulizia, la memoria liberata è sparsa in blocchi di dimensioni diverse, portando a frammentazione dell'Heap, che può rendere difficile l'allocazione di nuovi oggetti grandi.

algoritmo di Garbage Collection



Mark-and-Sweep Algorithm

Il Mark-and-Sweep si basa sull'idea di partire dalle radici (variabili attive nello Stack) e contrassegnare tutti gli oggetti raggiungibili. Ciò che rimane non contrassegnato è garbage.

Poiché l'operazione è complessa e costosa in termini di tempo, è spesso eseguita da un thread dedicato (il Daemon) che opera in background e si attiva quando l'Heap raggiunge una soglia di riempimento.

- Assume tag bits associated with data
- Need list of locations named by program
- Algorithm:

L'algoritmo richiede che ogni cella di memoria nell'Heap abbia un piccolo spazio aggiuntivo, un "Tag Bit" (o "Bit di Marca"), che può assumere il valore 0 o 1. Questo bit serve per contrassegnare temporaneamente lo stato di raggiungibilità dell'oggetto.

Si riferisce a tutti i luoghi in cui il programma conserva i riferimenti validi agli oggetti allocati nell'Heap. Queste locazioni costituiscono l'insieme di partenza, o Radici (Roots), del processo di marcatura. Le Radici del GC si trovano in aree di memoria che non fanno parte dell'Heap e la cui lifetime è gestita da meccanismi diversi (Stack).

- Set all tag bits to 0.

All'inizio del ciclo di GC, tutti i tag bit di tutti gli oggetti nell'Heap vengono resettati a 0. Significato: Inizialmente, si assume che tutti gli oggetti nell'Heap siano garbage.

- Start from each location used directly in the program. Follow all links, changing tag bit to 1

- Consider as garbage all cells with tag = 0

Il GC scorre l'intera Heap, esaminando tutti gli oggetti.

- Se un oggetto ha Tag Bit = 0, è garbage. L'algoritmo lo dealloca e lo aggiunge alla lista della memoria disponibile.
- Se un oggetto ha Tag Bit = 1, è ancora in uso e viene mantenuto. (Alcuni algoritmi resettano il tag a 0 per il ciclo successivo in questa fase).

Questa è la fase di marcatura, in cui si identificano gli oggetti non garbage.

- Punto di Partenza (Radici): Le "locations used directly in the program" sono le Radici del GC. Queste sono tutti i puntatori che si trovano nei Record di Attivazione attivi sullo Stack.
- Processo: L'algoritmo esegue una visita (come una visita in profondità o in ampiezza) a partire da ciascuna radice. Ogni volta che incontra un oggetto non ancora marcato:
> Imposta il suo Tag Bit a 1.
> Segue tutti i puntatori contenuti in quell'oggetto e ripete il processo.
- Risultato: Al termine di questa fase, solo gli oggetti raggiungibili (quelli a cui il programma può potenzialmente accedere) avranno il Tag Bit = 1.

cioè arrivato in un oggetto dell'Heap, visita tutti i riferimenti che ha all'interno dell'oggetto perché a sua volta gli oggetti al suo interno possono essere acceduti tramite l'oggetto acceduto all'inizio

L'algoritmo Mark-and-Sweep usa questa lista come punto di partenza per la Fase di Mark (Marcatura):
- Il GC esamina ogni locazione in questa lista (ogni Radice).
- Se la locazione contiene un puntatore, segue il link fino all'oggetto nell'Heap.
- Marca l'oggetto come "raggiungibile" (Tag Bit = 1).
- Ripete la marcatura ricorsivamente per tutti i puntatori contenuti in quell'oggetto, creando un grafo di raggiungibilità.

L'idea centrale di questo algoritmo è che ogni blocco di memoria allocato nell'Heap tiene traccia di quanti puntatori attivi (riferimenti) puntano ad esso.
Il Reference Counter: Ogni blocco di dati allocato nell'Heap ha un campo aggiuntivo, il Reference Counter, per memorizzare il numero intero di puntatori attuali che lo indirizzano.
Inizializzazione: Quando un dato è allocato per la prima volta nell'Heap (es. con new), il suo contatore viene inizializzato a 0 (o 1, se l'allocazione stessa crea il primo riferimento).

algoritmo di Garbage Collection

2

Reference Counting algorithm

Non è strettamente necessario un processo Daemon separato per la deallocazione con il Conteggio dei Riferimenti (Reference Counting) perché l'algoritmo opera in modo incrementale e immediato (in linea) con l'esecuzione del programma.

A differenza del Mark-and-Sweep, che è un processo a interruzione (stop-the-world), il Conteggio dei Riferimenti è un processo incrementale e distribuito.

Each datum in memory has an associated reference counter

- When a datum is allocated in memory, initialize the counter to 0
- When a pointer to a datum is set, increments the counter
- When a pointer to a datum is reset, decrement the counter
- When the counter turns to be 0, the datum is garbage

Gli aggiornamenti del contatore avvengono in linea con l'esecuzione del programma, ogni volta che un puntatore viene modificato. Questo richiede che il compilatore generi codice speciale per queste operazioni.

Si può risolvere applicando le Reference Weak:

- Quando un programmatore crea una struttura dati che è intrinsecamente ciclica (ad esempio, un nodo figlio che punta al nodo genitore in un albero), uno dei due puntatori nel ciclo (solitamente il puntatore che chiude il ciclo, come il figlio che punta al genitore) viene designato come Riferimento Debole.
- Quando il ciclo si crea, il puntatore forte contribuisce all'Reference Counter, ma il puntatore debole non contribuisce.
- Quando l'intero ciclo diventa irraggiungibile dalle radici del programma, tutti i riferimenti forti esterni (cioè, dallo Stack o da oggetti non ciclici) spariscono.
- L'oggetto A (e B) raggiungono un Contatore di Riferimenti pari a 0 (perché il riferimento debole non è contato), e l'intero ciclo viene deallocato correttamente.

Quando il contatore scende a 0, il blocco di memoria viene immediatamente deallocato e aggiunto alla lista della memoria disponibile (o restituito al gestore dell'Heap)

Esempio: L'Oggetto A punta a B, e l'Oggetto B punta ad A (si tengono in vita insieme).

Vantaggio:

- Incrementale: La GC avviene in piccoli passi, evitando le lunghe pause del Mark-and-Sweep.
- Locale: La deallocazione è rapida e avviene non appena l'oggetto diventa garbage.
- Nessun Lungo Stop: Non interrompe l'esecuzione del programma per lunghe fasi di scansione.

Svantaggio:

- Overhead: Richiede un overhead significativo per ogni operazione di puntatore (incremento/decremento).
- Impossibile Gestire i Cicli: Non riesce a liberare la memoria per gli oggetti che formano cicli di riferimenti. Anche se gli oggetti nel ciclo non sono raggiungibili dalle Radici, ognuno ha un contatore ≥ 1 , impedendo la deallocazione.
- Maggiore Spazio: Richiede spazio di memoria aggiuntivo in ogni oggetto per memorizzare il contatore.

Notes

La memoria allocata a un programma in esecuzione è suddivisa in quattro aree principali, ognuna con responsabilità e caratteristiche diverse in termini di dimensione ed accesso

- 1 • The code area contains object code
- For most languages, fixed size and read only
codice macchina generato dal compilatore.
Essendo immutabile, può essere condivisa tra processi multipli che eseguono lo stesso programma
Dimensione Fissa (determinata in fase di compilazione/linking) e solitamente Sola Lettura (Read Only).
- 2 • The static area contains data (not code) with fixed addresses (e.g., global data)
- Fixed size, may be readable or writable
esempio:
Dimensione Fissa (determinata in fase di linking), può essere Leggibile o Scrivibile (a seconda che si tratti di dati globali inizializzati o costanti). Questi dati hanno la lifetime che coincide con l'intera durata del programma.
- 3 • The stack contains an AR for each currently active procedure
- Each AR usually fixed size (for a given procedure), contains locals
Ogni AR contiene le variabili locali della procedura e le informazioni di controllo
La dimensione del singolo AR è Fissa per quella specifica procedura (determinata dal compilatore). L'area Stack nel suo complesso è Dinamica e cresce/diminuisce in modo LIFO. Gestisce l'allocazione automatica di variabili e il flusso di controllo delle chiamate a funzione, grazie alla sua natura LIFO (Last-In, First-Out).
- 4 • Heap contains all other data
Contiene tutti gli altri dati, ovvero la memoria allocata dinamicamente dal programmatore (es. tramite malloc, new) la cui lifetime non è legata alla funzione che l'ha creata. Dimensione Dinamica (cresce e si riduce in base alle richieste del programma). È sempre Scrivibile. Permette la creazione di strutture dati complesse (alberi, liste) e oggetti la cui durata non segue la struttura rigida dello Stack.

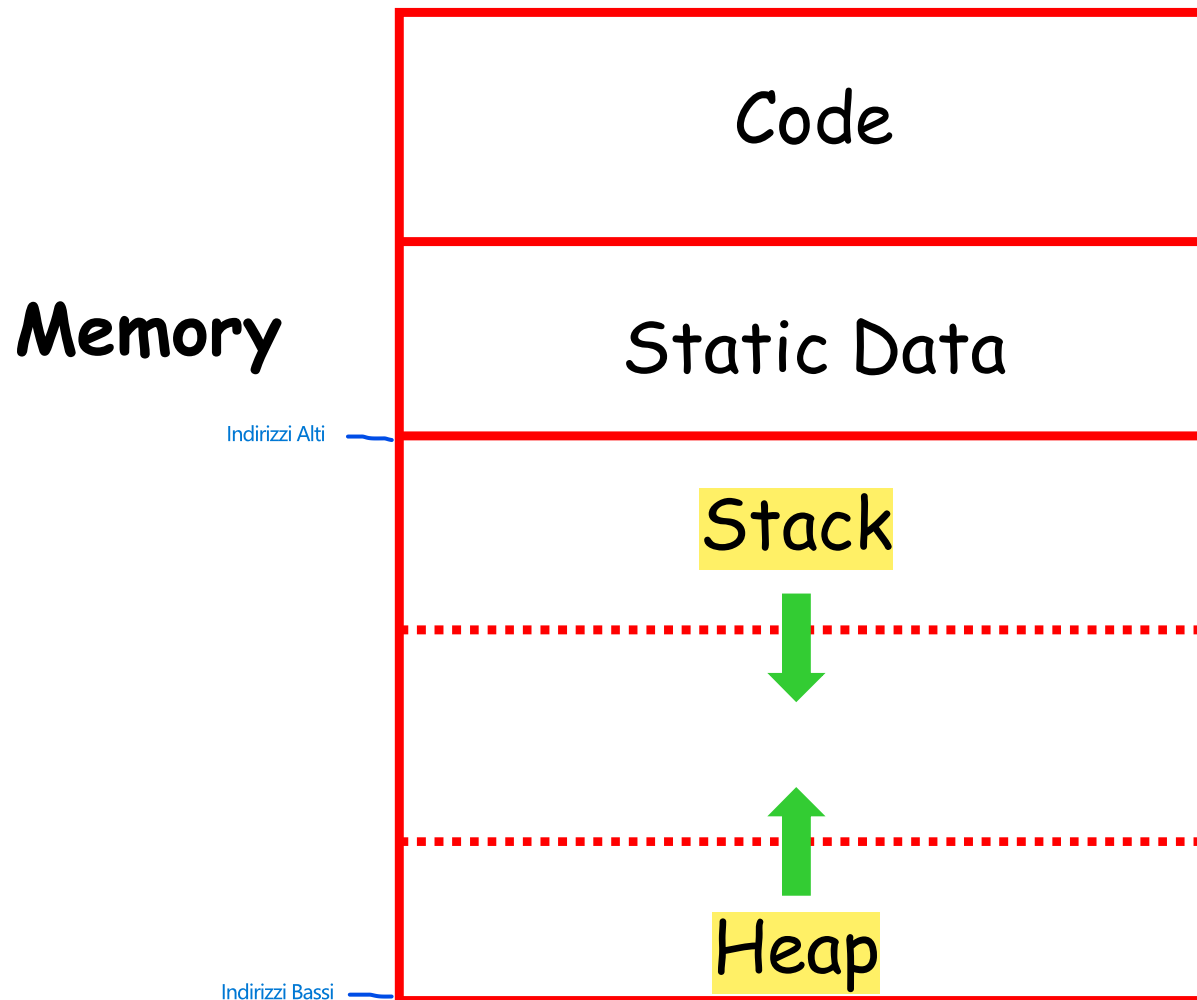
Notes (Cont.)

si cercano di ri-utilizzare
i buchi del Heap

- Both the heap and the stack grow
- Must take care that they don't grow into each other
- Solution: start heap and stack at opposite ends of memory and let them grow towards each other

La memoria del programma finisce quando
Heap e Stack Pointer coincidono/collidono

Memory Layout with Heap



modello di macchina che semplifica notevolmente il processo

Code Generation for Stack Machine

Lecture Outline

- Stack machines
- The MIPS assembly language
- A simple source language
- Stack-machine implementation of the simple language

Stack Machines

Questo modello è ideale per la fase di Generazione di Codice perché la traduzione di espressioni in notazione postfissa è diretta e richiede una gestione dei dati molto semplificata rispetto a una macchina a registri.

- **A simple evaluation model**

Principio: L'esecuzione è modellata come una serie di istruzioni che manipolano uno Stack (Pila) centrale, utilizzato per contenere i dati necessari per l'operazione successiva.
Ordine Postfisso: Le Stack Machines spesso eseguono le operazioni usando la notazione polacca inversa (o postfissa). Ad esempio, per calcolare $A+B$, si pusha A, si pusha B, e poi si esegue l'operazione ADD.

- **No variables or registers**

Tutta la manipolazione dei dati e l'archiviazione dei risultati intermedi avvengono direttamente e unicamente sullo Stack dei Valori. Le istruzioni non hanno bisogno di specificare gli operandi, rendendole molto compatte.

- **A stack of values for intermediate results**

Lo Stack funge da spazio di lavoro temporaneo per i risultati parziali delle espressioni.

Example of a Stack Machine Program

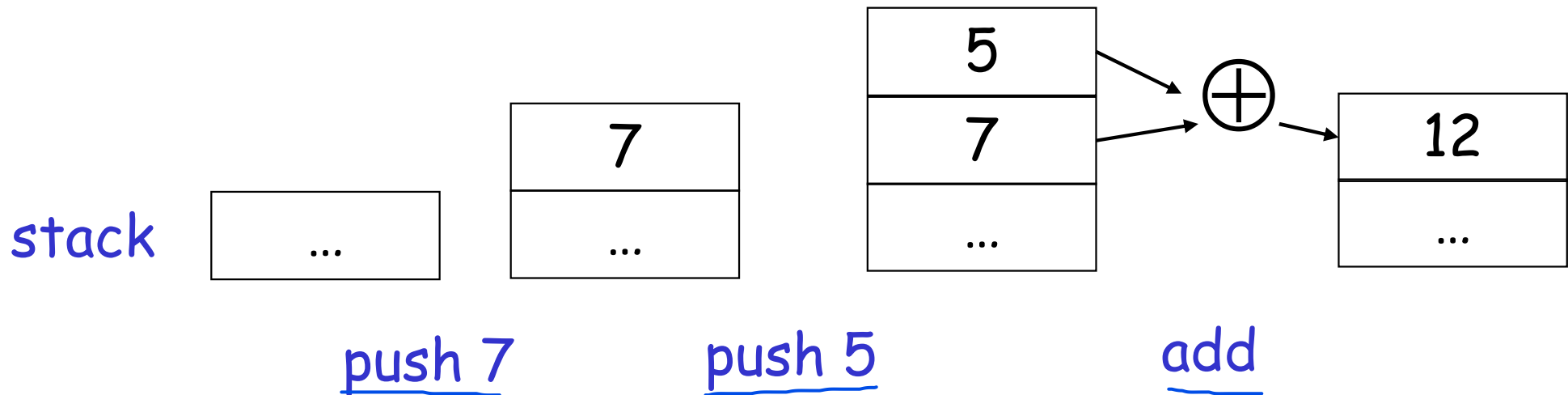
- Consider two instructions
 - push i - place the integer i on top of the stack
 - add - pop two elements, add them and put the result back on the stack
- A program to compute $7 + 5$:

push 7

push 5

add

Stack Machine. Example



- Each instruction:
 - Takes its operands from the top of the stack
 - Removes those operands from the stack
 - Computes the required operation on them
 - Pushes the result on the stack

Why Using a Stack Machine ?

- Locazioni Fisse: Ogni operazione prende i suoi operandi dallo stesso posto (la cima dello Stack) e mette il risultato nello stesso posto (la nuova cima dello Stack).
- Semplicità: Questa uniformità elimina la necessità per il compilatore di prendere decisioni complesse di gestione delle risorse, come:
 - > Scegliere quale dei molti registri della CPU usare per un operando.
 - > Gestire l'allocazione dei registri o il salvataggio dei registri in memoria.
- Risultato: Il processo di traduzione da un'Espressione in codice macchina diventa diretto e meccanico, seguendo la notazione postfissa.

- Each operation takes operands from the same place and puts results in the same place

Schema di Compilazione Uniforme

- This means a uniform compilation scheme

- And therefore a simpler compiler

Poiché lo schema di compilazione è uniforme, il codice del compilatore (in particolare il generatore di codice) è intrinsecamente più semplice e meno soggetto a errori. Il compilatore non deve implementare algoritmi di ottimizzazione complessi per i registri, riducendo drasticamente la complessità del back-end.

- **Invariant:**

Dopo aver calcolato un'espressione, lo Stack è come prima, con il risultato aggiuntivamente inserito in cima.

- Significato: Se si calcola l'espressione E, il codice generato per E deve assicurare che la dimensione dello Stack finale sia uguale alla dimensione iniziale più 1 (per il risultato).
- Beneficio: Questa proprietà rende la generazione di codice per le espressioni ricorsiva e facile da implementare, poiché la compilazione di un sotto-albero di espressione non interferisce con la gestione dello Stack richiesta per gli altri sotto-alberi.

- After computing an expression the stack is as before with result additionally pushed on top

Why Using a Stack Machine ?

- **Location of the operands is implicit**
 - Always on the top of the stack

La posizione degli operandi è implicita, trovandosi sempre in cima allo Stack.
- **No need to specify operands explicitly**

Non è necessario specificare esplicitamente gli operandi nell'istruzione, né è necessario specificare la posizione del risultato.
- **No need to specify the location of the result**
- **Instruction “add” as opposed to “add r_0, r_1, r_2 ” (three-address code)**
 - ⇒ Smaller encoding of instructions
 - ⇒ More compact programs

L'istruzione add (per una Stack Machine) è più piccola in termini di codifica rispetto a un codice a tre indirizzi come add r_0, r_1, r_2 . Ciò porta a un'codifica delle istruzioni più piccola e a programmi più compatti.
- **This is one reason why Java Bytecodes use a stack evaluation model**

Questo modello di valutazione basato sullo Stack è la ragione per cui i Java Bytecodes utilizzano un modello di valutazione a pila. Tale modello astratto facilita la portabilità e la verifica del codice, rendendolo ideale per macchine virtuali (come la JVM).

Computing Expressions with Stack Machine

Remember the invariant:

Dopo aver calcolato una (sotto)espressione, lo stack è come prima con il risultato aggiuntivamente inserito in cima.

After computing a (sub)expression the stack is as before with result additionally pushed on top

La Regola di Generazione Ricorsiva

For an operation $op(e_1, e_2)$:

Il Contenuto dello Stack deve essere preservato durante la generazione del calcolo della sotto-espressione

Genera il codice per calcolare la sotto-espressione e_1

Compute e_1 (at the end the stack is as before with just e_1 result additionally pushed on top)

Genera il codice per calcolare la sotto-espressione e_2

Compute e_2 (at the end the stack is as before with just e_2 result additionally pushed on top)

Genera l'istruzione op

Compute op (pops 2 values and pushes op result)

Questo processo è noto come traversata in post-ordine (post-order traversal) dell'Albero Sintattico Astratto (AST) dell'espressione, ed è la tecnica standard per la generazione di codice per le Stack Machines.

Il risultato finale dell'espressione $13 - (7 + 5)$ è 1, che è l'unico valore rimasto sullo Stack. Questo esempio dimostra perfettamente l'efficacia dell'invariante della Macchina a Pila: ogni volta che una sotto-espressione è completata (add o sub), lo Stack si riduce e rimane solo il risultato.

A Bigger Example: $13 - (7 + 5)$

Code

push 13

push 7

push 5

add

sub

Stack

<init>

13, <init>

Valuta il primo operando dell'operazione esterna (la sottrazione). L'invariante è mantenuto: 13 è in cima.

7, 13, <init>

5, 7, 13, <init>

Valutano ricorsivamente i sotto-operandi dell'espressione (7+5).

12, 13, <init>

Estrae 5.
Estrae 7.
Calcola $7 + 5 = 12$.
Inserisce 12.
Invariante Mantenuto: Lo Stack è ora come era prima di valutare (7+5), ma con il risultato 12 in cima (accanto al 13 che era sotto).

1, <init>

Estrae 12 (l'operando destro della sottrazione).
Estrae 13 (l'operando sinistro della sottrazione).
Calcola $13 - 12 = 1$.
Inserisce 1.

Notes

- It is very important that the stack is preserved across the evaluation of a subexpression
 - Stack before the evaluation of $7 + 5$ is: $13, \langle \text{init} \rangle$
 - Stack after the evaluation of $7 + 5$ is: $12, 13, \langle \text{init} \rangle$
 - After popping the result (12) of the subexpression $(7+5)$ the stack must be as before

The Stack Machine can be optimized

Poiché la cima dello Stack è l'area più frequentemente accessibile, questi accessi continui alla memoria sono il collo di bottiglia dell'efficienza (quindi lento nell'esecuzione perché fa accessi in memoria che sono più lenti rispetto a quelli fatti sui registri).

- The add instruction does 3 memory operations
 - Two reads and one write to the stack
 - The top of the stack is frequently accessed
- Idea: keep the top of the stack in a register (called **accumulator**)
 - Register accesses are faster
- The “add” instruction is now
$$\text{acc} \leftarrow \text{acc} + \text{top_of_stack}$$
 - Only one memory operation!
- For simplicity we will not use this optimization

Idea: Mantenere la cima dello Stack in un registro dedicato, chiamato Accumulatore (Acc). Gli accessi ai registri sono intrinsecamente più veloci degli accessi alla memoria.

L'operazione add ora richiede solo una operazione di memoria (la lettura del secondo operando, `top_of_stack`), mentre l'operando in cima e il risultato rimangono nel registro veloce (l'Accumulatore).

L'obiettivo è creare un compilatore che genera codice per il modello Stack Machine senza accumulatore e poi mostrare come questo codice astratto viene tradotto in istruzioni eseguibili su un processore standard a registri.

Le istruzioni del modello Stack Machine (Macchina a Pila) vengono tradotte in istruzioni MIPS Assembly per poter essere eseguite su un processore reale. Questa traduzione è necessaria perché la Stack Machine è un modello astratto di valutazione (basato sullo Stack degli operandi), mentre MIPS è un'architettura hardware a registri concreta.

From Stack Machines to Assembly language

Il compilatore genera inizialmente istruzioni per una Macchina a Pila (senza l'ottimizzazione dell'Accumulatore). Questo codice è concettualmente semplice e uniforme.

1. We consider a compiler that generates code for a stack machine (without accumulator)

2. We want to run the resulting code on some processor

Il codice generato deve essere eseguito su un processore reale. Il processore scelto come esempio prototipico è il MIPS: È un'architettura RISC (Reduced Instruction Set Computer) ampiamente utilizzata nell'istruzione e nella ricerca per la sua chiarezza e semplicità. A differenza della Stack Machine, il MIPS è un'architettura a registri, dove la maggior parte delle operazioni aritmetiche opera su valori memorizzati nei registri interni della CPU, non direttamente in memoria.

We consider the prototypical processor known as **MIPS** (there exist simulators for several real architectures)

- Microprocessor without Interlocked Pipeline Stages

3. More precisely:

Il passo finale è l'implementazione (o emulazione) delle istruzioni della Macchina a Pila utilizzando le istruzioni e i registri del MIPS. Questo processo richiede di:

- Designare i Registri: Decidere quali registri MIPS saranno utilizzati per gestire lo Stack.

- Traduzione: Tradurre ogni singola istruzione stack-based in una sequenza di istruzioni MIPS che manipolano quei registri e la memoria.

Passando a MIPS, abbandoniamo l'astrazione e entriamo nei dettagli di come la memoria e i registri vengono effettivamente gestiti in hardware.

- We implement stack machine instructions using MIPS instructions and registers

MIPS Assembly

MIPS architecture

- Prototypical Reduced Instruction Set Computer (RISC) architecture

Le architetture RISC utilizzano un numero ridotto di istruzioni semplici e uniformi, che possono essere eseguite molto rapidamente e in modo efficiente dalla pipeline del processore (la pipeline è l'architettura che divide l'esecuzione di un'istruzione in una serie di stadi, permettendo al processore di lavorare su più istruzioni contemporaneamente in stadi diversi).

- Arithmetic operations use registers for operands and results

Tutte le operazioni aritmetiche e logiche devono usare i registri per i loro operandi e per memorizzare i risultati. Non possono operare direttamente sui valori in memoria.

- Must use load and store instructions to use operands and results in memory

Per accedere o salvare i dati nella memoria principale, si devono usare esplicitamente istruzioni di caricamento (load) e memorizzazione (store)

- 32 general purpose registers (32 bits each)

↳ We will use \$sp and \$t0, \$t1, \$t2 (temporary registers)

↳ Registri temporanei a scopo generale che useremo per caricare i dati dalla memoria, eseguire i calcoli e memorizzare i risultati prima di scriverli nuovamente nello Stack.

Simulating a Stack Machine...

- The stack is kept in memory
- The stack grows towards lower addresses
 - (in previous presentation of “memory management” assume lower addresses to be at the bottom)
- Stack pointer is kept in MIPS register `$sp`
 - The top element of the stack is at address `$sp`

A Sample of MIPS Instructions

Carica una parola (32 bit) dalla memoria. L'indirizzo è calcolato sommando il valore del registro base (reg1) e l'offset immediato. Il dato caricato viene salvato nel registro destinazione (reg0).

lw reg₀ offset(reg₁)

- Load 32-bit word from address ^{presente in} reg₁ + offset into reg₀

Addizione tra registri.

add reg₀ reg₁ reg₂

- reg₀ ← reg₁ + reg₂

Memorizza una parola (32 bit) dalla memoria. Il contenuto del registro sorgente (reg0) viene scritto nella memoria all'indirizzo calcolato come (reg1 + offset).

sw reg₀ offset(reg₁)

- Store 32-bit word in reg₀ at address ^{presente in} reg₁ + offset

Addizione immediata unsigned.

addiu reg₀ reg₁ imm

- reg₀ ← reg₁ + imm

Caricamento Immediato. Carica un valore costante (imm) direttamente nel registro destinazione (reg0).

li reg imm

- reg ← imm

Some Useful Macros

Definisce le macro in Assembly MIPS che ci permetteranno di scrivere codice per la Stack Machine in modo più conciso e leggibile. Le macro sono essenzialmente delle scorciatoie che espandono una singola istruzione in una sequenza di istruzioni MIPS.

Assumiamo che ogni elemento sullo Stack sia una parola (4 byte), quindi il cambiamento di sp è sempre di 4.

- We define the following abbreviations

Inserisce il contenuto del registro t in cima allo Stack.

- **push \$t**

```
addiu $sp $sp -4
```

```
sw $t 0($sp)
```

Rimuove il valore in cima allo Stack, ma non lo salva in un registro.

- **pop**

```
addiu $sp $sp 4
```

Estrae il valore in cima e lo carica nel registro t.

- **\$t ← pop**

```
lw $t 0($sp)
```

```
addiu $sp $sp 4
```

Rimuove n elementi dalla cima dello Stack.

- **pop*n**

```
addiu $sp $sp z
```

with $z = 4 * n$ (pop n times)

MIPS Assembly. Example.

- The stack-machine code for $7 + 5$ in MIPS:

push 7

Carica la costante 7 nel registro t0 e poi esegue la macro push t0. Questo mette il valore 7 in cima allo Stack in memoria.

li \$t0 7
push \$t0

push 5

Carica la costante 5 nel registro t0 e poi esegue la macro push t0. Lo Stack ora contiene [5, 7, ...]

li \$t0 5
push \$t0

add

Simula l'operazione add:

1. pop 5 in t2.
2. pop 7 in t1.
3. Calcola $t0 = 7 + 5 = 12$.
4. Esegue la macro push t0 per mettere il risultato 12 in cima.

\$t2 ← pop
\$t1 ← pop
add \$t0 \$t1 \$t2
push \$t0

In ADD, metto prima in T2 poi in T1 perché l'inserimento nello stack porta ad avere alla fine, prima il secondo operatore poi il primo.

A Small Language

- Tipi di Dati: Supporta solo gli interi (integers).
- Operazioni: Include operazioni su interi (integer operations) (addizione, sottrazione).
- Funzioni: Supporta funzioni globali (global functions) con almeno un parametro. (Il che suggerisce che non supporta l'annidamento di funzioni).

- A language with integers, integer operations, and global functions with at least one parameter

$$P \rightarrow D ; P \mid D$$

$$D \rightarrow \text{fun id(ARGS) = E}$$

Una dichiarazione di funzione definisce una funzione con un identificatore (id), una lista di argomenti (ARGS) ed un corpo che è un'espressione (E).

$$\text{ARGS} \rightarrow \text{id, ARGS} \mid \text{id}$$

$$E \rightarrow \underline{\text{int}} \mid \underline{\text{id}} \mid \underline{\text{if } E_1 = E_2 \text{ then } E_3 \text{ else } E_4} \\ \mid E_1 \pm E_2 \mid E_1 = E_2 \mid \underline{\text{id}(E_1, \dots, E_n)}$$

espressione condizionale

non ci sono
scope innestati

Chiamata di funzione, con n
espressioni come argomenti.

A Small Language (Cont.)

- The first function definition f is the “main” routine
- Running the program on input i means computing f(i)
- Program for computing the Fibonacci numbers:

fun fib(x) = if x = 1 then 0 else
if x = 2 then 1 else
fib(x - 1) + fib(x - 2)

Code Generation Strategy

La strategia si concentra sul mantenere l'Invariante della Stack Machine per garantire che la generazione del codice per le espressioni sia corretta e composizionale (cioè, il codice per le sotto-espressioni può essere combinato facilmente).

- For each expression e we generate MIPS code that:

- Computes the result of e and pushes it on the stack

Calcola e PUSH: Calcola il risultato dell'espressione e e lo inserisce (PUSH) sulla cima dello Stack.

- Preserves the contents that were on the stack before evaluating e

Preserva lo Stack Precedente: Preserva il contenuto dello Stack che era presente prima dell'inizio della valutazione di e .

Se lo Stack iniziale era $S = [..., X, Y]$, dopo l'esecuzione di $cgen(e)$, il nuovo Stack è $S' = [..., X, Y, risultato(e)]$.

← the only difference is that, after evaluating e , the stack additionally contains the result of e on top

L'unica differenza è che, dopo la valutazione di e , lo Stack contiene in aggiunta il risultato di e in cima.

- We define a code generation function $cgen(e)$ whose result is the code generated for e

Formalmente, definiamo una funzione di generazione di codice, $cgen(e)$, il cui risultato è la sequenza di istruzioni MIPS generate per valutare l'espressione e secondo la strategia definita. Questo approccio ci permette di procedere ricorsivamente. Ad esempio, per $E1 + E2$, possiamo invocare $cgen(E1)$ e $cgen(E2)$ e poi aggiungere l'istruzione ADD (tradotta in MIPS).

1 Code Generation for Constants

- The code to evaluate a constant (expression "i", e.g. "5") simply pushes it on the stack:

Per mantenere l'invariante della Stack Machine (calcolare il risultato e inserirlo in cima allo Stack, preservando il resto dello Stack), il codice generato per una costante è diretto: deve solo inserire il valore di i sulla cima dello Stack.

↔ `cgen(i) = li $t0 i`

Carica il valore costante i direttamente in un registro temporaneo t0.

`push $t0`

Espande a:

- addiu sp, sp, -4 (Alloca spazio nello Stack)
- sw t0, 0(sp) (Scrivi il valore dal registro t0 in cima allo Stack)

- Note that this also preserves the stack, as required

2 Code Generation for Add

$cgen(e_1 + e_2) =$

$cgen(e_1)$

Genera il codice per valutare l'espressione e_1 . Il risultato di e_1 viene messo in cima allo Stack.

$cgen(e_2)$

Genera il codice per valutare l'espressione e_2 . Il risultato di e_2 viene messo sopra il risultato di e_1 .

$\$t2 \leftarrow \text{pop}$

Estrae (POP) il risultato di e_2 (l'operando destro) nel registro temporaneo $t2$.

$\$t1 \leftarrow \text{pop}$

Estrae (POP) il risultato di e_1 (l'operando sinistro) nel registro temporaneo $t1$.

$\text{add } \$t0 \ \$t1 \ \$t2$

Esegue l'addizione MIPS sui registri

$\text{push } \$t0$

Inserisce (PUSH) il risultato finale dalla $\$t0$ in cima allo Stack.

- Possible optimization: Put the result of e_1 immediately in register $\$t1$?

È possibile ottimizzare, inserendo il risultato di e_1 immediatamente nel registro $t1$?

'' Optimized'' Code for Add. Wrong!!!

- Optimization: Put the result of e_1 immediately in $\$t1$?

```
cgen( $e_1 + e_2$ ) =  
  cgen( $e_1$ )  
   $\$t1 \leftarrow \text{pop}$   
  cgen( $e_2$ )  
   $\$t2 \leftarrow \text{pop}$   
  add  $\$t0 \$t1 \$t2$   
  push  $\$t0$ 
```

Violazione della Regola di Preservazione dei Registri:

- La regola fondamentale è: Non aspettarti mai che il valore di un registro sia preservato attraverso il codice generato!
- Problema: La sequenza tenta di salvare il risultato di e_1 nel registro temporaneo $t1$ e assume che quel valore rimanga in $t1$ mentre viene eseguita la funzione $\text{cgen}(e_2)$.
- Pericolo: La funzione $\text{cgen}(e_2)$ è un'intera sequenza di codice che valuta un'espressione. Se e_2 è complessa (es. $e_2 = A \times B$), il codice generato per e_2 farà un uso intensivo dei registri temporanei ($t0$, $t1$, $t2$, ecc.) per calcoli intermedi e PUSH/POP. Il codice di $\text{cgen}(e_2)$ probabilmente sovrascriverà il registro $t1$.

L'unica risorsa su cui $\text{cgen}(e_2)$ può contare per la preservazione dei dati è lo Stack in memoria. Qualsiasi tentativo di salvare un valore a lungo termine in un registro temporaneo ($t0$, $t1$, $t2$) che verrà utilizzato ricorsivamente è un errore. Il codice non ottimizzato (quello corretto) funziona proprio perché usa due PUSH e due POP, assicurando che i risultati intermedi di e_1 ed e_2 siano sempre salvati in modo sicuro sullo Stack in memoria.



Non aspettarsi mai che il valore di un registro, generato da una code gen di una espressione, venga preservato dopo le code gen di altre espressioni (Invece, bisogna usare lo Stack per preservare i calcoli intermedi, non i registri che sono temporanei)

- Try to generate code for : $13 + (7 + 5)$
- Never expect the value of a register to be preserved across generated code!

Questo è possibile grazie all'invariante: poiché sia e_1 che e_2 lasciano il loro risultato in cima allo Stack e preservano tutto il resto, il codice di e_2 può essere semplicemente eseguito subito dopo quello di e_1 .

Code Generation Notes

La strategia di generazione del codice per la Stack Machine è intrinsecamente ricorsiva e segue la struttura dell'espressione che si sta traducendo.

- The code for $+$ is a template with “holes” for il codice generato per la valutazione delle sotto-espressioni code for evaluating e_1 and e_2
- Stack-machine code generation is recursive
- Code for $e_1 + e_2$ consists of code for e_1 and e_2 glued together e consiste nel codice generato per e_1 seguito dal codice generato per e_2 ed infine dalle istruzioni per l'operazione $+$ stessa.
- Code generation is performed bottom-up by a recursive traversal of the AST

Ordine di Esecuzione

La generazione di codice viene eseguita bottom-up (dal basso verso l'alto) mediante una traversata ricorsiva dell'AST (Albero Sintattico Astratto). Questo corrisponde a una traversata in post-ordine (post-order traversal): si generano prima i codici per i nodi figli (e_1 e e_2) e poi il codice per il nodo padre ($+$). Questa strategia assicura che, nel codice finale, gli operandi di un'espressione (e_1 ed e_2) siano calcolati e spinti nello Stack prima che l'operazione ($+$) li estraiga ed esegua il calcolo.

3

Code Generation for Sub

- New instruction: `sub reg0 reg1 reg2`
 - Implements $\text{reg}_0 \leftarrow \text{reg}_1 - \text{reg}_2$

`cgen(e1 - e2) =`

`cgen(e1)`

`cgen(e2)`

`$t2 ← pop`

`$t1 ← pop`

`sub $t0 $t1 $t2`

`push $t0`

L'ordine di POP è cruciale per la sottrazione (e per la divisione) perché, sebbene e₂ sia l'ultimo a essere spinto, il suo valore (il sottraendo) deve essere usato come operando destro (t2) nella sottrazione.

4 Code Generation for Conditional

- We need flow control instructions

Le istruzioni di controllo del flusso ci permettono di alterare la sequenza lineare di esecuzione del codice, saltando a etichette (label) specifiche.

Salto Condizionale (Branch Equal)

- New instruction: `beq reg1 reg2 label`
 - Branch to label if $reg_1 = reg_2$

Esegue un salto condizionale all'etichetta specificata (label) se il valore del registro `reg1` è uguale al valore del registro `reg2`.

Salto Incondizionato (Branch)

- New instruction: `b label`
 - Unconditional jump to label

Esegue un salto incondizionato all'etichetta specificata (label). L'esecuzione riprende immediatamente da quella posizione nel codice.

Code Generation for If (Cont.)

Le label (etichette) nel contesto della generazione di codice MIPS Assembly, in particolare per implementare costrutti come l'istruzione condizionale if-then-else, indicano una posizione specifica (un indirizzo) all'interno del codice. Le label sono essenziali per il controllo del flusso e agiscono come target per le istruzioni di salto.

$\text{cgen}(\text{if } e_1 = e_2 \text{ then } e_3 \text{ else } e_4) =$

$\text{cgen}(e_1)$

$\text{cgen}(e_2)$

$\$t2 \leftarrow \text{pop}$

$\$t1 \leftarrow \text{pop}$

beq \$t1 \$t2 true_branch

$\text{cgen}(e_4)$

b end_if

Dopo aver eseguito il ramo else, viene eseguito un salto incondizionato per bypassare il ramo then.

true_branch: Questa etichetta è la destinazione del salto condizionale.

$\text{cgen}(e_3)$

end_if: Questa etichetta segna il punto in cui entrambi i rami (vero e falso) convergono. L'esecuzione continua da qui.

Se i risultati non sono uguali (beq fallisce), il controllo prosegue nel ramo "else".

Actually label names need to be freshly generated by $\text{cgen}()$!

e.g. try to generate code in the case e_3 is again an if-then-else

Se i risultati sono uguali (risultato(e_1)=risultato(e_2)), il controllo salta immediatamente all'etichetta true_branch.

Attualmente i nomi delle etichette devono essere generati ex novo da $\text{cgen}()$!

- Perché: Se le etichette (true_branch ed end_if) fossero nomi fissi, ed e_3 fosse a sua volta un'espressione if-then-else (nidificata), anche essa tenterebbe di usare le stesse etichette.
- Problema: Questo causerebbe un conflitto di nomi e salti non voluti, portando a codice errato.
- Soluzione: Quando $\text{cgen}()$ incontra un if, deve generare etichette uniche (ad esempio, usando un contatore globale: true_branch_1, end_if_1, true_branch_2, ecc.) per ogni istanza dell'espressione condizionale.

The Activation Record

- Code for function calls and function definitions depends on the layout of the activation record
- A very simple AR suffices for this language:
 - The result of a function call is simply pushed on top of the stack at the end of its execution (as for any other expression)
 - No need to reserve a space for the result in the AR
 - The activation record holds parameter values
 - For $f(x_1, \dots, x_n)$ the activation record holds $x_n, \dots, x_1$ values
 - These are the only variables in this language

Nel nostro linguaggio semplice, i parametri sono le uniche variabili gestite dal compilatore. L'AR deve quindi garantire l'accesso a questi valori durante l'esecuzione del corpo della funzione. I parametri sono spesso spinti (PUSHed) sullo Stack in ordine inverso ($x_n, \dots, x_1$) prima della chiamata effettiva.

Dobbiamo anche memorizzare il Control Link che è un puntatore che punta all'AR del chiamante. Questo link viene utilizzato per:

- Ripristinare lo Stack: Durante il ritorno da una funzione, il Link di Controllo è essenziale per ripristinare correttamente lo Stack Pointer (\$sp) al suo valore precedente la chiamata (ovvero, la posizione dello Stack prima che l'AR del chiamante fosse creato).
- Tracciare il Flusso: Definisce la catena dinamica delle chiamate.

Dobbiamo memorizzare l'indirizzo di ritorno. Questo è l'indirizzo della prossima istruzione nel codice del chiamante che deve essere eseguita una volta che la funzione chiamata ha terminato il suo compito. Senza questo, il processore non saprebbe dove riprendere l'esecuzione.

The Activation Record (Cont.)

- We need to store the return address
- We also need the Control Link (next slide)
 - Access Link is not needed because we do not have nested declarations. We just have:
 - local parameter declarations and
 - functions that are all declared in the global scope that is allocated statically

Nel nostro linguaggio, abbiamo solo due tipi di allocazioni di variabili:

- Parametri Locali: Dichiarazioni di parametri locali all'interno di una funzione.
- Funzioni Globali: Tutte le funzioni sono dichiarate nello scope globale

Il Control Link, che punta all'AR del chiamante, viene gestito attraverso il fp stesso:

- Link come Vecchio fp: Il Link di Controllo del chiamante (l'AR genitore) è l'indirizzo della sua posizione di riferimento, che è il suo Frame Pointer.
- Memorizzazione nell'AR: È necessario memorizzare il valore del \$fp del chiamante (il vecchio \$fp) nell'AR corrente.
- Funzione: In questo modo, quando la funzione corrente termina, il valore del registro \$fp può essere ripristinato al valore del chiamante, ristabilendo il contesto di accesso ai dati del chiamante.
- In sintesi, il \$fp non solo facilita l'accesso ai dati, ma è anche il mezzo tramite cui viene implementato il Control Link.

The Activation Record (Cont.)

Il Frame Pointer (fp) è un registro della CPU utilizzato per fornire un punto di riferimento stabile all'interno del Record di Attivazione corrente.

È utile avere un puntatore a una posizione fissa di riferimento nell'AR corrente.

utile
It's handy to have a pointer to a reference position in the current Activation Record

- This pointer lives in register `$fp` (frame pointer)
 - The reference position is part of AR layout design
 - Here we take `$fp` to point at position of the first argument
- `$fp` used by generated code to locate AR elements, e.g. parameters, based on offsets

Parametri, indirizzo di ritorno e Link di Controllo vengono localizzati utilizzando offset positivi o negativi rispetto all'indirizzo stabile memorizzato in `$fp`.

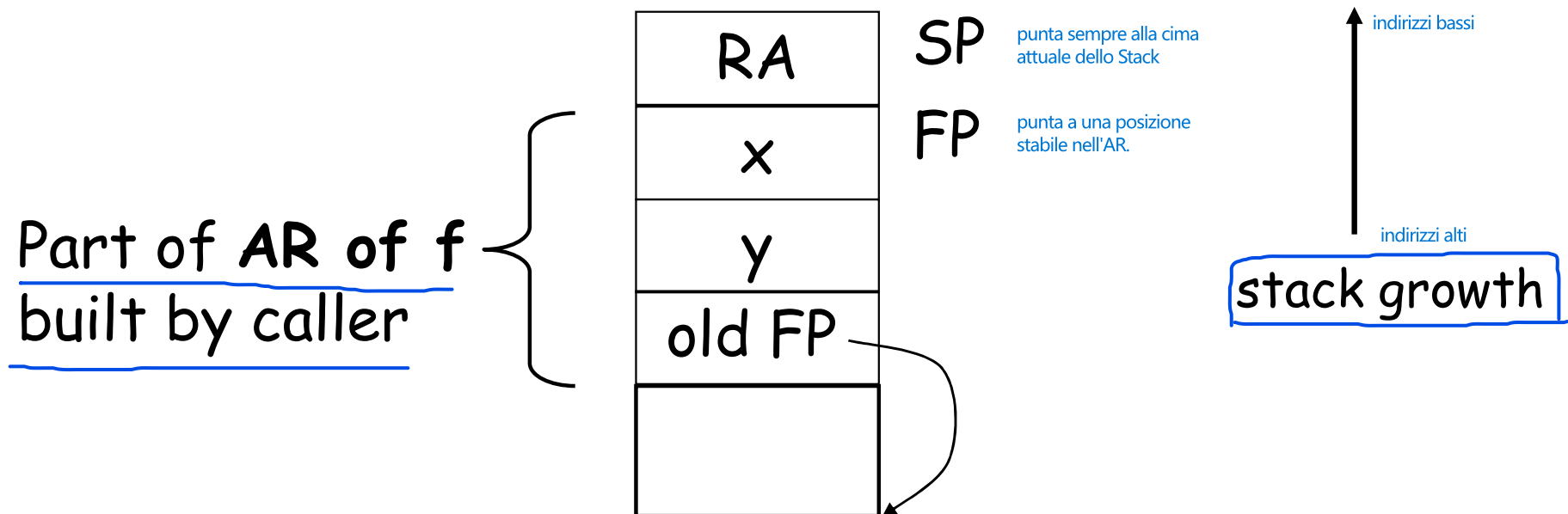
- We need to store `$fp` of the caller (Control Link) in the AR, so that we can restore it
 - in general for linking to an AR we use the address of its reference position

The Activation Record

Per il nostro linguaggio minimale (che non ha funzioni annidate), l'AR richiede solo tre componenti chiave per gestire il controllo e i dati:

- Control Link (il vecchio \$fp).
- Valori dei Parametri.
- Indirizzo di Ritorno (RA).

- Summary:
 - For this language, an AR with the caller's frame pointer (Control Link), the parameters, and the return address (RA) suffices
- Consider a call to $f(x,y)$. The AR will be:



5

Code Generation for Function Call

Collaborazione tra il codice del chiamante e il codice della funzione chiamata per impostare e smantellare correttamente l'AR

- The calling sequence is the instructions (of both caller and callee) to set up a function invocation

La calling sequence è l'insieme di istruzioni (sia del chiamante che del chiamato) necessarie per impostare (set up) correttamente l'invocazione di una funzione. Obiettivo: Assicurare che tutti i dati e i puntatori di controllo (AR, parametri, indirizzo di ritorno) siano posizionati correttamente nello Stack prima che l'esecuzione della funzione chiamata inizi.

- New instruction: jal label

Jump to label, save address of next instruction in \$ra

viene utilizzata per la chiamata a subroutine (funzione)

- instruction name means: jump and link (call to subroutine)

Il registro \$ra è essenziale: memorizzando l'indirizzo di ritorno, la funzione chiamata può, alla sua conclusione, utilizzare l'istruzione jr \$ra (Jump Register) per ritornare correttamente al codice del chiamante, completando l'invocazione.

Code Generation for Function Call (Cont.)

$cgen(f(e_1, \dots, e_n)) =$

$\leftarrow$ push \$fp

cgen(e_n)

...

cgen(e_1)

jal f_entry

The caller saves its value of the frame pointer

Then it saves the actual parameters in reverse order

The caller saves the return address in register \$ra

The AR so far is $4*n+4$ bytes long

Il chiamante salva il suo valore del Frame Pointer. Questo valore salvato è il Link di Controllo del nuovo Record di Attivazione (AR), che servirà al chiamato per ripristinare il contesto del chiamante al momento del ritorno.

Esegue un salto incondizionato alla label f_entry (l'inizio del codice della funzione f). L'istruzione jal salva automaticamente l'indirizzo di ritorno (l'istruzione successiva) nel registro ra (Return Address). Questo completa l'AR per quanto riguarda l'Indirizzo di Ritorno.

Dopo l'esecuzione di questi passi, la parte dell'AR costruita dal chiamante (i parametri e l'old FP) è nello Stack. Poiché ci sono n parametri e un fp salvato, l'AR finora ha una dimensione di $4n+4$ byte (assumendo 4 byte per ogni elemento). Dopo il salto, la funzione chiamata deve completare l'AR.

6

Code Generation for Function Definition

- New instruction: `jr reg`
 - Jump to address in register `reg`

Salta all'indirizzo contenuto nel registro specificato

`cgen(fun f( $x_1, \dots, x_n$ ) = e) =`

`f_entry:` Questa è la label (etichetta) a cui salta l'istruzione `jal` del chiamante.

`move $fp $sp`

`push $ra` Il chiamante salva l'Indirizzo di Ritorno (che è stato posizionato in registro `ra` dal `jal`) nello Stack. Questo completa l'AR.

`cgen(e)` Viene eseguito il codice per valutare l'espressione `e`, che costituisce il corpo della funzione. Il risultato di `e` viene spinto in cima allo Stack.

`$t0 ← pop` Il risultato della funzione viene estratto (POP) e memorizzato temporaneamente nel registro `t0`.

`$ra ← pop` L'Indirizzo di Ritorno (`ra`) viene ripristinato dal suo slot nello Stack al registro `ra`.

`pop *n` → Vengono rimossi i parametri (in numero `n`) dallo Stack.

`$fp ← pop` Il vecchio Frame Pointer viene ripristinato dal suo slot nello Stack al registro `fp`. Questo ripristina il contesto del chiamante.

`push $t0` → Il risultato della funzione, temporaneamente salvato in `t0`, viene riposizionato (PUSHed) in cima allo Stack. Questo è il risultato finale della chiamata `f(...)` per il chiamante.

`jr $ra` → Esegue un salto incondizionato all'indirizzo contenuto nel registro `ra`. Il controllo ritorna all'istruzione successiva al `jal` del chiamante.

Content of `$sp` is copied into `$fp` Questo stabilisce il nuovo Frame Pointer per l'AR corrente, puntando alla cima dell'AR parziale costruito dal chiamante

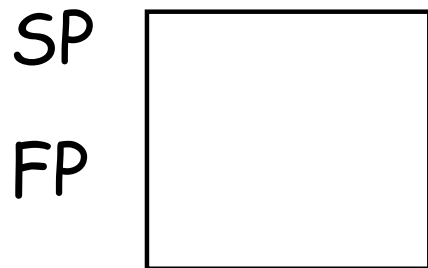
Note: The frame pointer points to first argument (not to bottom of frame)

`$t0` is a register used to temporarily hold the return value

- The callee pops the return value and removes all parts of frame of `f` from the stack

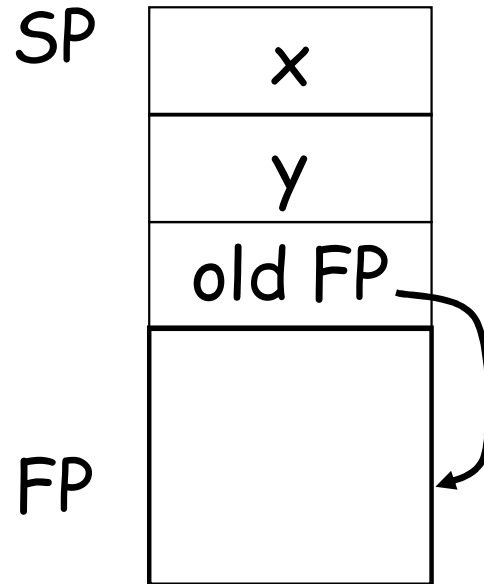
Calling Sequence. Example for $f(x,y)$.

Before call



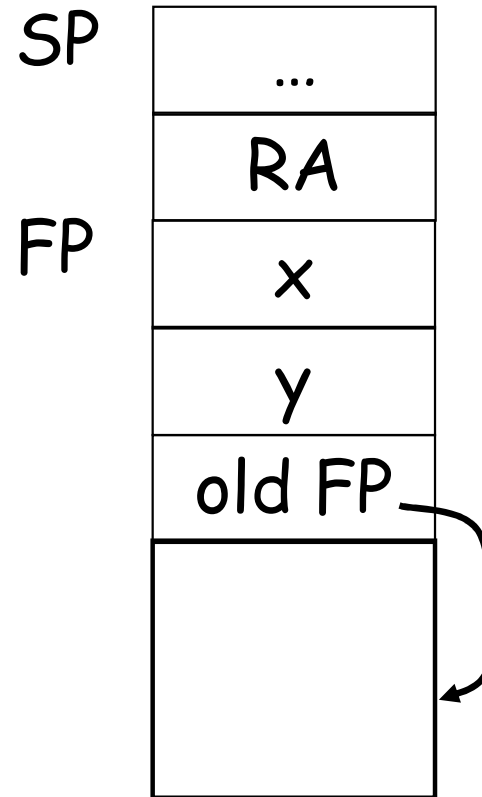
On entry

Chiamata Funzione



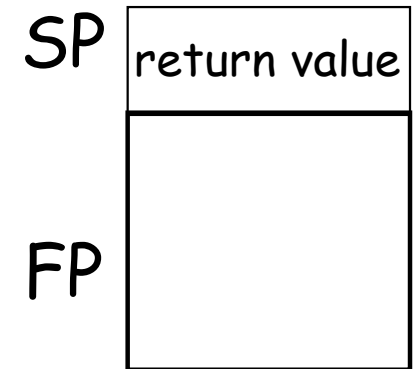
In body

Nell'esecuzione del body della funzione



After call

Dopo la chiamata a funzione
(ritorno da funzione)



La ragione principale per cui gli AR non sono adiacenti è che lo Stack cresce (ovvero, il puntatore \$sp si sposta) quando i risultati intermedi delle espressioni vengono salvati. Lo Stack serve contemporaneamente per:
Memorizzare i Record di Attivazione (AR) e Memorizzare i risultati temporanei delle espressioni (Stack degli Operandi).

Activation Records not adjacent on stack!

Because the stack grows when intermediate results are saved, Activation Records are not adjacent on stack!

- Example: $\text{fun } f(x) = 3 + g(5)$

- Upon execution 3 is on the stack between the AR of f and the AR of g

Poiché gli AR sono separati da dati intermedi, questo rafforza la necessità del Control Link e del Frame Pointer:

- Link di Controllo/old FP: Sebbene sp sia mobile, il Link di Controllo (il vecchio fp) nell'AR di g punterà comunque all'indirizzo stabile del Frame Pointer dell'AR di f , ignorando i dati intermedi.
- Accesso ai Dati: I parametri e le variabili locali all'interno di un AR (come l'AR di f) sono sempre accessibili tramite offset rispetto al loro Frame Pointer fp , indipendentemente da quanti dati temporanei si trovino sopra o tra gli AR.

7 Code Generation for usage of Parameters

- Usages of Parameters/Variables are the last construct we consider
- In this language the “variables” of a function are just its parameters
 - They are in current AR (we don't have nested scopes)
 - Allocated by the caller (that pushes their value)

Poiché non abbiamo scope annidati, non è necessario un Link di Accesso per trovarli; si trovano sempre nell'AR corrente.

I loro valori sono stati allocati (spinti) sullo Stack dal chiamante come parte della sequenza di chiamata.

- Problem: because, again, the stack grows when intermediate results are saved, the variables are not at a fixed offset from \$sp

- Example: $\text{fun } f(x) = 3+x$

Per risolvere questo problema, utilizziamo il Frame Pointer (fp).
Il fp è un puntatore stabile all'interno dell'AR (punta al Control Link/vecchio fp).
Tutti i parametri hanno un offset fisso e noto rispetto a fp, indipendentemente da quanti risultati intermedi siano stati spinti sopra di essi.
Il codice generato per l'uso di un parametro dovrà quindi usare fp come registro base.

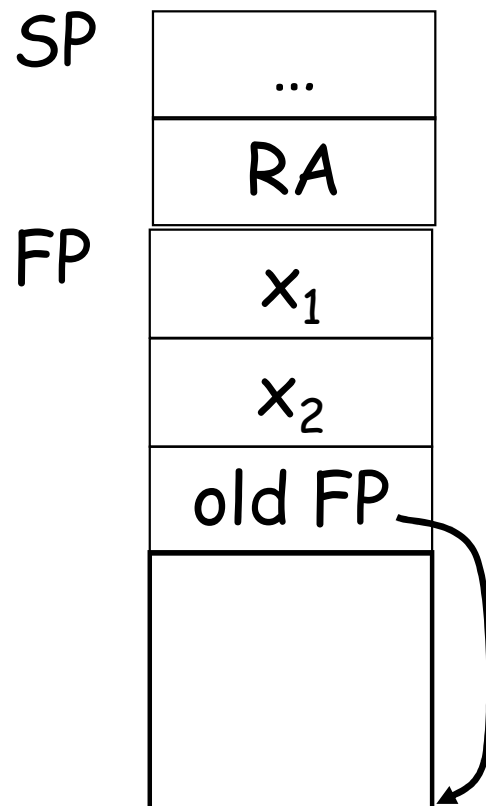
Code Generation for Variables (Cont.)

- **Solution:** use the frame pointer `$fp`
 - in our AR layout it always points to the first variable
-  Let x_i be the i^{th} ($i = 1, \dots, n$) formal parameter of the function for which code is being generated

Per accedere all'i-esimo parametro, x_i , dobbiamo calcolare l'offset fisso rispetto a fp .

Code Generation for Variables (Cont.)

- Example: For a function $\text{fun } f(x_1, x_2) = e$ the activation record and frame pointer are set up as follows:



x_1 is at $\$fp$

x_2 is at $\$fp + 4$

- Thus, based on **offset** of x_i :

$\text{cgen}(x_i) = \text{lw } \$t0 \text{ } z(\$fp)$
 $\text{push } \$t0$

with offset $z = 4 \cdot (i-1)$

- Info about the **offset** z of each parameter name **needs to be included** in its symbol table entry

La struttura del Record di Attivazione (AR) e il codice che lo gestisce sono inseparabili:

- Integrazione Compilatore/Runtime: Il layout del Record di Attivazione deve essere progettato insieme al generatore di codice.
- Dipendenza dal Linguaggio: Il design dell'AR (la posizione di parametri, Link di Controllo, Indirizzo di Ritorno, e variabili locali) è cruciale, poiché definisce come il codice Assembly MIPS, basato sul Frame Pointer (\$fp) e sugli offset, accede ai dati.

La generazione di codice per le espressioni segue un metodo algoritmico ben definito:

- Traversata Ricorsiva: La generazione di codice viene eseguita tramite una visita ricorsiva dell'AST.
- Post-Ordine: Questa visita è essenzialmente bottom-up, dove prima si genera il codice per le sotto-espressioni (e1, e2) e poi si aggiungono le istruzioni per l'operazione genitore (ad esempio, add, sub).
- Invariante della Stack Machine: L'intero processo si basa sull'invariante che ogni sotto-espressione calcolata lascia il suo risultato in cima allo Stack, preservando il resto dello Stack.

Summary

- The activation record layout must be designed together with the code generator
- Code generation can be done by recursive traversal of the AST
- To simplify the presentation we have not discussed the Access Links!
 - But they can be easily added (as already discussed in “memory management”):
 - check the difference in the nesting level between the caller code and the declaration of the called function, and follow the already settled access links accordingly

Code Generation for Object-Oriented Languages

Two Issues

- How are objects represented in memory?
- How is dynamic dispatch implemented?
 - sometimes called dynamic binding

Object Layout Example

```
class A {  
    int a = 0;  
    int d = 1;  
    int f() { return a = a + d }  
}
```

```
class B extends A {  
    int b = 2;  
    int f() { return a } //override  
    int g() { return a = a - b }  
}
```

```
class C extends A {  
    int c = 3;  
    int h() { return a = a * c }  
}
```

Object Layout (Cont.)

- Fields `a` and `d` are inherited by classes `B` and `C`
- All methods in all classes refer to `a`
- For `A` methods to work correctly in `A`, `B`, and `C` objects, field `a` must be in the same “place” in all objects

Quando un oggetto viene creato (istanziato):
- Il layout corrispondente viene creato (allocato) nello heap (memoria dinamica).
- Lo spazio occupato dall'oggetto persisterà finché non sarà più necessario.
- L'oggetto viene eventualmente rimosso dal Garbage Collector (GC), che è il sottosistema runtime responsabile della deallocazione automatica della memoria non più utilizzata.
In sintesi, il compilatore traduce l'accesso ai campi in un semplice calcolo di indirizzo ($\text{base} + \text{offset}$), rendendo l'accesso ai dati molto efficiente e diretto.

Object Layout (Cont.)

An object is like a struct in C. The reference
`foo.field`

is an index into a `foo` struct at an **offset**
corresponding to `field`

L'informazione necessaria per calcolare l'indirizzo del campo è gestita dal compilatore: L'offset del campo deve essere inserito nella Symbol Table entry (la voce della tabella dei simboli) per quel campo.

Un oggetto in un linguaggio orientato agli oggetti (OO) viene mappato in una sezione contigua di memoria, in modo simile a un record o una struct.

- Object is laid out in a contiguous memory section

Accesso ai Campi tramite Offset

- Each field stored at a fixed **offset** in object

- offset needs to be inserted in field symbol table entry

On object creation, the corresponding layout is
instantiated in the heap (it will be eventually
removed from the garbage collector)

Subclasses

Observation: Given a layout for class *A*, a layout for subclass *B* can be defined by:

- extending the layout of *A* with additional slots for the fields with new names defined in *B*

Leaves the layout of *A* "part" unchanged

- layout of *B* is an extension of it

Therefore: objects of class *B* preserve offsets of fields declared by *A*

Dynamic Dispatch

- Consider again our example

```
class A {  
    int a = 0;  
    int d = 1;  
    int f() { return a = a + d }  
}
```

```
class B extends A {  
    int b = 2;  
    int f() { return a }  
    int g() { return a = a - b }  
}
```

```
class C extends A {  
    int c = 3;  
    int h() { return a = a * c }  
}
```

Dynamic Dispatch Example

Il Dynamic Dispatch è la meccanica che garantisce che, anche se si accede a un oggetto tramite un puntatore di un tipo base (tipo statico), il codice eseguito sia quello del metodo più specifico (tipo dinamico/runtime).

- e.g() with e having static type B
 - calls method g of B
- e.f() with e having static type A
 - calls method f of A if e yields an A or C object (f is inherited in the case of C)
 - calls method f of B if e yields a B object (f is overridden)

tipo dinamico

- The implementation of methods and dynamic dispatch strongly resembles the implementation of fields

L'implementazione dei metodi e del dynamic dispatch assomiglia fortemente all'implementazione dei campi.

Proprio come un campo viene trovato ad un offset fisso all'interno del blocco di memoria dell'oggetto, anche i metodi vengono trovati tramite un meccanismo basato su offset.

In pratica, questo si realizza tramite la V-Table (Virtual Method Table). Ogni oggetto OO contiene un puntatore nascosto a una V-Table statica per la sua classe.

- Oggetto: Contiene un puntatore alla V-Table della sua classe.

- V-Table: È un array di puntatori a funzione (gli indirizzi di memoria dei metodi).

- Dynamic Dispatch: Per chiamare e.f(), il compilatore cerca l'offset di f all'interno della V-Table. A runtime, segue il puntatore alla V-Table dell'oggetto e, e poi usa quell'offset per trovare e saltare all'indirizzo corretto della funzione, garantendo che sia chiamato il metodo di A, B o C appropriato.

Dispatch Tables

- Every class has a fixed set of methods (including inherited methods)
- A dispatch table indexes these methods
 - An array of method addresses
 - A method *f* lives at a fixed offset in the dispatch table for a class and all of its subclasses

La Dispatch Table è la chiave per la corretta esecuzione dinamica dei metodi, garantendo che sia chiamato il codice specifico per il tipo runtime dell'oggetto.

1. Struttura e Contenuto

- Set Fisso di Metodi: Ogni classe possiede un set fisso di metodi (che include i metodi definiti nella classe stessa più tutti i metodi ereditati dalle sue superclassi).

- Dispatch Table: La Dispatch Table è un array di indirizzi di metodi. Ogni elemento dell'array è un puntatore (indirizzo) alla posizione in memoria in cui risiede il codice Assembly di un metodo.

2. L'Invariante Cruciale - Offset Fissi

Il punto fondamentale per far funzionare il Dynamic Dispatch è l'uso degli offset:

- Offset Fissi: Un metodo specifico *f* si trova a un offset fisso all'interno della Dispatch Table sia per la classe che lo definisce sia per tutte le sue sottoclassi.

> Esempio: Se il metodo `draw()` si trova all'indice 5 nella V-Table della classe base `Shape`, allora `draw()` si troverà all'indice 5 anche nella V-Table di `Circle` e `Square`.

- Vantaggio: Il compilatore può determinare, in fase di compilazione, l'offset di un metodo (ad esempio, l'indice 5 per `draw()`). A runtime, il codice può usare questo offset fisso per trovare l'indirizzo della funzione corretta nella V-Table dell'oggetto, anche se il metodo è stato sovrascritto.

3. Implementazione (Sintesi)

- Ogni oggetto in memoria include, come primo campo, un puntatore alla V-Table della sua classe runtime.

- Per chiamare `e.f()`:

> Il compilatore sa l'offset `$z$` di *f*.

> A runtime, il codice usa il puntatore alla V-Table dell'oggetto `$e$`.

> Il codice salta all'indirizzo memorizzato nella V-Table all'indice `$z$`.

> Poiché la V-Table della sottoclasse contiene l'indirizzo del metodo sovrascritto, si esegue il metodo corretto.

Dispatch Table Example

Fisso per ogni metodo
di quella classe

<u>Offset</u>	0	4
<u>Class</u>		
A	f of A	
B	f of B	g
C	f of A	h

- The dispatch table for class A has only 1 method
- The tables for B and C extend the table for A to the right
- Because methods can be overridden, the address for f is not the same in every class, but is always at the same offset

La chiave per il Dynamic Dispatch è l'invariante dell'Offset:

Poiché i metodi possono essere sovrascritti (come f in B), l'indirizzo della funzione per f non è lo stesso in ogni classe, ma è sempre allo stesso offset (Offset 0).

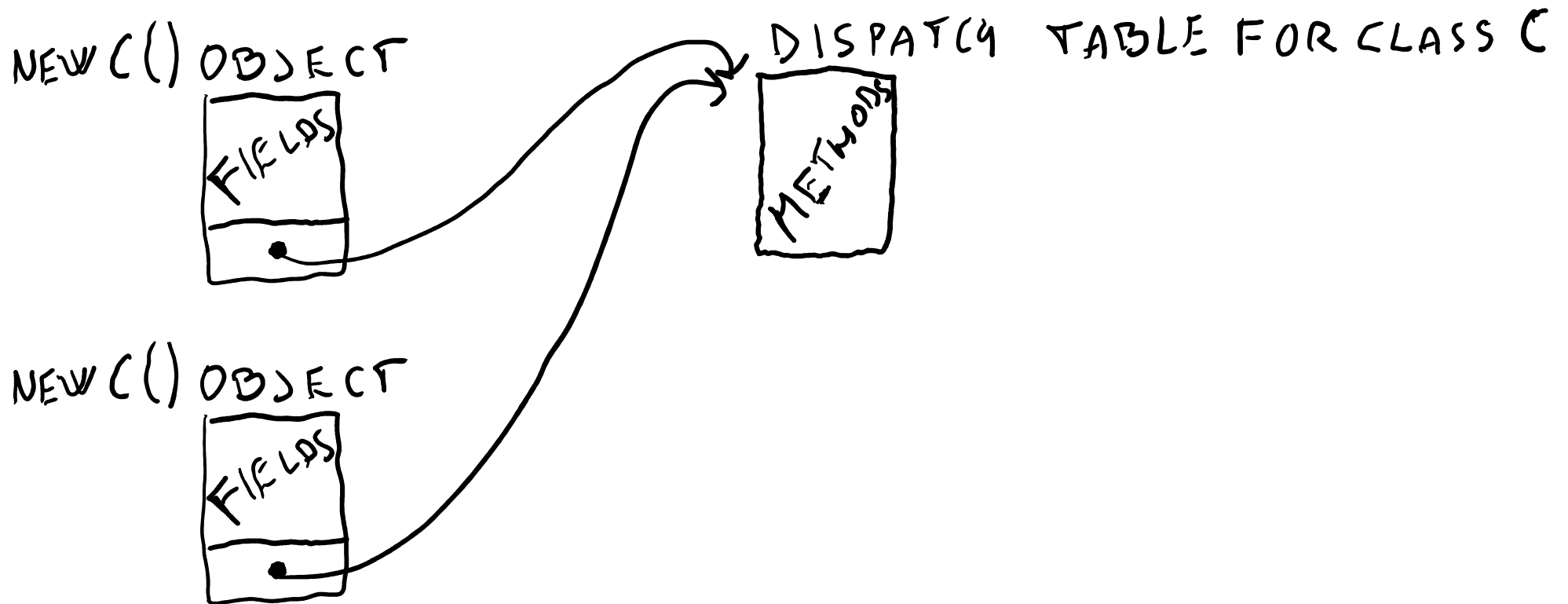
Questo permette al compilatore di generare un codice Assembly MIPS efficiente:

- Compilazione: Se il compilatore vede una chiamata a e.f(), esso genera un codice che cerca all'Offset 0.

- Esecuzione: A runtime, indipendentemente dal fatto che l'oggetto sia di tipo A, B o C, l'accesso all'Offset 0 della sua V-Table troverà l'indirizzo corretto (sia esso f di A o f di B), implementando così il polimorfismo.

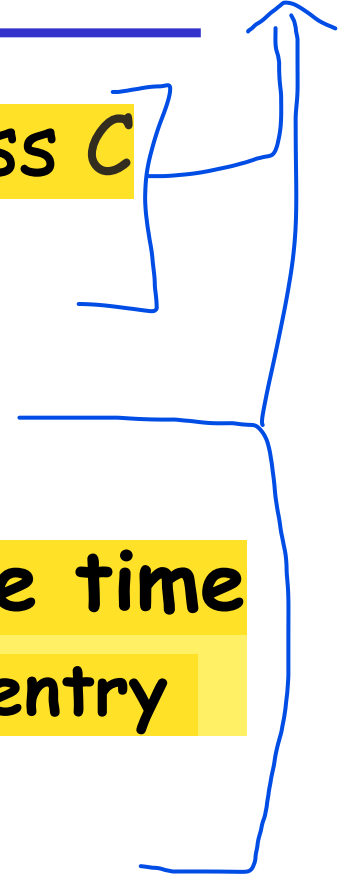
Using Dispatch Tables

- The dispatch pointer in an object of class *C* points to the dispatch table for class *C*



Using Dispatch Tables

1. Il Puntatore di Dispatch nell'Oggetto (Runtime)
 - Puntatore all'Oggetto: Ogni oggetto in memoria (ad esempio, un'istanza della classe C) contiene, tipicamente come primo campo, un puntatore di dispatch (o V-Pointer).
 - Destinazione: Questo puntatore punta all'indirizzo in memoria della Dispatch Table (V-Table) specifica della sua classe runtime (C).
2. Assegnazione dell'Offset (Compile Time)
 - Offset Fissi: Ad ogni metodo f della classe C (inclusi i metodi ereditati e/o sovrascritti) viene assegnato un offset fisso (O_f) all'interno della sua Dispatch Table.
 - Symbol Table: Questo offset (O_f) viene determinato in fase di compilazione e, analogamente agli offset delle variabili/campi, viene inserito nella Symbol Table entry del metodo f della classe C.

- The dispatch pointer in an object of class C points to the dispatch table for class C
 - Every method f of class C is assigned an offset O_f in the dispatch table at compile time
 - The offset is inserted in the symbol table entry of method f of class C as usual
- 

Using Dispatch Tables (Cont.)

- To implement a dynamic dispatch $e.f()$ we
 - 1 - Let O_f be the offset of the method f in the dispatch-table associated to the static type of e
 - 2 - Evaluate e , obtaining an object o (that could be of any subclass)
 - 3 [- Let D be the dispatch-table of o
- Execute the method pointed by $D[O_f]$

Il Dynamic Dispatch è un processo in tre fasi che combina informazioni determinate in fase di compilazione con dati specifici dell'oggetto ottenuti in fase di esecuzione.

1. Fase di Compilazione (Offset Statico)

- Identificare l'Offset: Il compilatore determina l'offset O_f (ad esempio, un indice z) del metodo f nella Dispatch Table.

- Base Statica: Questo offset è ricavato dal tipo statico di e . Poiché l'offset è fisso in tutta la gerarchia di ereditarietà, questo valore O_f è stabile e corretto per tutte le sottoclassi.

2. Fase di Esecuzione (Oggetto Dinamico)

- Valutazione dell'Espressione: L'espressione e viene valutata.

- Risultato: Il risultato è un oggetto o un indirizzo in memoria.

- Tipo Runtime: L'oggetto o potrebbe essere un'istanza del tipo statico di e o di qualsiasi sua sottoclasse.

3. Fase di Dispatch (Selezione del Metodo)

- Trovare la D-Table: Si accede all'oggetto o per trovare il suo puntatore di dispatch, che punta alla Dispatch Table D specifica per la classe runtime di o .

- Esecuzione del Metodo: Viene eseguito il metodo puntato dall'elemento all'offset O_f nella Dispatch Table D :

> Esegui il metodo puntato da $D[O_f]$

Poiché D è la V-Table della classe runtime e O_f è l'indice corretto per f , l'indirizzo ottenuto è quello della versione più specifica del metodo f (quella sovrascritta), implementando così correttamente il Dynamic Dispatch.